Contents

1. Objective . . . 4

2. Reference Tools . . . 4

3. Approach . . . 4

4. Architecture . . . 5

4.1 Architecture diagram . . . 5
4.2 Components . . . 5
4.3 Project files . . . 6
4.4 The model, the API and the UI . . . 6
4.5 Docker Compose configuration . . . 6
4.6 Dockerfiles and dependency configuration . . . 7
4.7 Running the project . . . 7
4.8 Testing and issues resolved . . . 7

5. Source Code . . . 8

5.1 docker-compose.yml . . . 8
5.2 api/Dockerfile . . . 8
5.3 ui/Dockerfile . . . 9
5.4 api/app.py - Flask API . . . 9
5.5 ui/streamlit_app.py - Streamlit UI . . . 11
5.6 tests/ui_apptest.py - UI integration test . . . 13
5.7 tests/test_api.py - API tests . . . 14
5.8 api/train_model.py - model training . . . 15
5.9 api/requirements.txt . . . 17
5.10 ui/requirements.txt . . . 17
5.11 api/.dockerignore . . . 17
5.12 ui/.dockerignore . . . 18
5.13 task11_docker.py - Docker workflow runner . . . 18

6. Command Execution Log . . . 20

6.1 Verify Docker and Compose (Steps 1-2) . . . 20
6.2 Validate the Configuration and Build the Images (Steps 3-8) . . . 21
6.3 Start the Full Application with Docker Compose (Steps 9-12) . . . 22
6.4 Test the Application (Steps 13-17) . . . 23
6.5 Inspect the Containers and the Network (Steps 18-25) . . . 25
6.6 Resilience and Dependency Checks (Steps 26-28) . . . 26
6.7 Cleanup (Step 29) . . . 27

7. Screenshots and Observations . . . 28

7.1 Building and Starting the Application . . . 28
7.2 Container Status . . . 29
7.3 Image Sizes . . . 29
7.4 The Streamlit UI: Start Page . . . 29
7.5 Prediction in the UI . . . 30
7.6 Prediction from Pixel Values . . . 30
7.7 The API Directly . . . 31
7.8 API Test Suite . . . 31
7.9 UI Integration Test inside the ui Container . . . 31
7.10 Non-root Users and the Private Network . . . 32
7.11 Docker Desktop - Containers . . . 32
7.12 Docker Desktop - Images . . . 33
7.13 Stopping the API Container . . . 34
7.14 UI Behaviour while the API Is Down . . . 34

8. Testing Report . . . 35

8.1 Test summary . . . 35
8.2 API test results (api_test_results.json) . . . 35
8.3 UI integration test results (ui_integration_test.txt) . . . 36
8.4 Sample predictions through the API and through the UI . . . 36
8.5 Resilience tests (steps 26-28) . . . 36
8.6 Images, resources and results by approach area . . . 37

9. Deployment Readiness . . . 37

10. Observations . . . 38

11. Known Limitations 38
12. Conclusion 39

1. Objective

To containerize the complete deep learning solution including model, Flask API, and Streamlit UI. The deliverables are the Dockerized application (Sections 4, 5 and 6), the architecture diagram (Section 4.1) and the testing report (Section 8).

2. Reference Tools

The solution consists of a NumPy neural-network model, a **Flask** API served by gunicorn and a **Streamlit** user interface. **Docker Desktop** (v4.94.0 on macOS), the **Docker CLI** (29.8.2) and **Docker Compose** (v5.5.1) build and run the containers; a Python script records the command, output and exit code of every step.

Item	Value from the recorded run
Docker Desktop / Engine / Compose	v4.94.0 / 29.8.2 / v5.5.1
Base image of both containers	python:3.12-slim (Python 3.12.15, Debian arm64)
API container	Flask 3.1.3, Werkzeug 3.1.9, gunicorn 26.2.0, NumPy 2.5.3 (9 pinned packages)
UI container	Streamlit 1.65.0, requests 2.34.2, pandas 3.0.6, NumPy 2.5.3 (36 pinned packages)
Ports	API: container 5000, host 8000 (macOS uses host port 5000 for AirPlay); UI: 8501
Network	dl-app-net (private bridge network created by Compose)
Image architecture	arm64 (Apple Silicon)

3. Approach

The five approach steps of the task were carried out in order. Section, step and figure numbers refer to this report.

1. **Package all application components** - the model and the Flask API form the `api` container, the Streamlit UI forms the `ui` container, and `docker-compose.yml` ties both together (Sections 4.2-4.5).
2. **Configure dependencies** - separate pinned `requirements.txt` files for `api/` and `ui/`, configuration through the environment variable `API_URL`, health checks and start order (Sections 4.5-4.6).
3. **Build Docker images** - `docker compose build` creates `dl-api:1.0` and `dl-ui:1.0` with multi-stage Dockerfiles (steps 3-8, Figures 1 and 3).
4. **Perform testing** - API test suite, UI integration test inside the ui container, container-to-container check, inspection, restart and outage tests (steps 9-28, Figures 2 and 4-14, Section 8).
5. **Document architecture** - the architecture diagram (Section 4.1), the component descriptions (Section 4) and the deployment-readiness review (Section 9).

4. Architecture

The complete solution runs as two containers. A user opens the Streamlit UI in the browser (host port 8501). The UI container has no model: it sends HTTP requests to the API container by the service name `api` over the private network `dl-app-net`. The API container runs gunicorn and the Flask application, which evaluates the NumPy model and returns JSON. Test clients can reach the API directly on host port 8000.

4.1 Architecture diagram

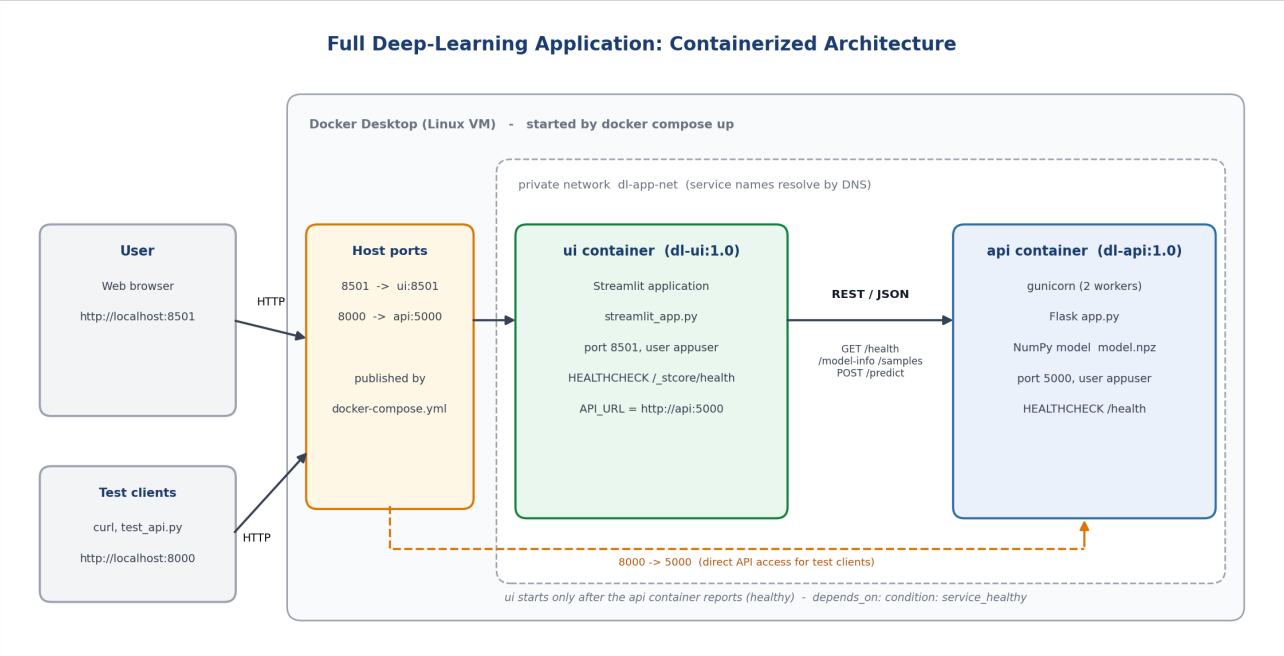


Diagram 1: Architecture of the containerized application (also delivered as the file architecture.png).

4.2 Components

Component	Container / image	Contents	Port
Model	inside the api container	`model/model.npz`: weights of a 64-64-10 neural network, metadata and 10 sample images (25,458 bytes)	-
Flask API	api / `dl-api:1.0`	Flask + gunicorn (2 workers), routes /health, /model-info, /samples, POST /predict	5000 (host 8000)
Streamlit UI	ui / `dl-ui:1.0`	Streamlit application `streamlit_app.py`, calls the API at `http://api:5000`	8501
Orchestration	docker compose	`docker-compose.yml`: builds, networks, port mapping, health-based start order	-

4.3 Project files

File	Role
`docker-compose.yml`	Defines the services api and ui, the network dl-app-net, ports, environment and start order.
`api/app.py`	Flask API with input validation and JSON errors.
`api/train_model.py`	Trains the model in NumPy and writes `api/model/model.npz` (not part of the image).
`api/Dockerfile`, `api/requirements.txt`	Multi-stage image of the API; 9 pinned packages.
`ui/streamlit_app.py`	Streamlit UI: sidebar API status, sample digits, pixel-value input, about tab.
`ui/Dockerfile`, `ui/requirements.txt`	Multi-stage image of the UI; 36 pinned packages.
`tests/test_api.py`	13 black-box API checks (run on the host).
`tests/ui_apptest.py`	8 UI integration checks with Streamlit AppTest (run inside the ui container).
`task11_docker.py`	Runs the 29 Docker steps and records the real output.

4.4 The model, the API and the UI

The model is a fully connected network (64 inputs, 64 ReLU units, 10 softmax outputs; 4,810 parameters) trained from scratch in NumPy on the UCI handwritten-digits dataset; it reaches 97.22 % accuracy on 360 held-out images. The Flask API loads the weights once and offers the routes `/health`, `/model-info`, `/samples` and `POST /predict` (one image or up to 100 images); invalid input is answered with a JSON error and HTTP 400.

The Streamlit UI reads the API address from the environment variable `API_URL`. Its sidebar shows whether the API is reachable together with the model's parameters and accuracy. The tab *Sample digits* predicts a held-out sample, the tab *Enter pixel values* predicts a pasted list of 64 values, and the tab *About* describes the architecture. If the API is not reachable the UI shows an error message instead of failing.

4.5 Docker Compose configuration

Setting	Value	Purpose
`name`	dl-app	project name; groups both containers in Docker Desktop
`services.api`	build ./api, image dl-api:1.0, container dl-api, ports 8000:5000, restart unless-stopped	API and model
`services.ui`	build ./ui, image dl-ui:1.0, container dl-ui, ports 8501:8501, restart unless-stopped	Streamlit UI
`environment`	API_URL: http://api:5000	the UI finds the API by its service name
`depends_on`	api, condition: service_healthy	the UI starts only after the API reports (healthy)
`networks.default`	name: dl-app-net	private network with DNS names for the services

4.6 Dockerfiles and dependency configuration

Property	api image	ui image
Base image	python:3.12-slim	python:3.12-slim
Build	2 stages: builder (pip install --no-cache-dir --prefix=/install), runtime	2 stages: builder (same pip command), runtime
Dependencies	requirements.txt: 9 pinned packages	requirements.txt: 36 pinned packages
Application files	`app.py`, `model/`	`streamlit_app.py`
User	appuser (uid 10001)	appuser (uid 10001)
HEALTHCHECK	GET /health every 10 s	GET /_stcore/health every 10 s
Command	gunicorn --bind 0.0.0.0:5000, 2 workers	streamlit run streamlit_app.py --server.port=8501 --server.address=0.0.0.0 --server.headless=true
Environment	PYTHONDONTWRITEBYTECODE, PYTHONUNBUFFERED, PORT=5000	same, plus API_URL=http://api:5000 and Streamlit usage statistics off

4.7 Running the project

- Install Docker Desktop, start it and wait until it shows *Engine running*. Host ports 8000 and 8501 must be free.
- Start the whole application with `docker compose up -d --build`, open `http://localhost:8501` for the UI and `http://localhost:8000/health` for the API, and stop it with `docker compose down`.
- For the full recorded workflow run `python3 task11_docker.py`; it writes `docker_command_log.txt` and `docker_results.json`.

4.8 Testing and issues resolved

Issue found	Cause	Outcome
Host port 5000 cannot be used on the Mac	macOS reserves port 5000 for AirPlay Receiver.	The API container listens on 5000 and is published on host port 8000; all calls work.
The UI must not start before the API is ready	The UI reads the API status when it loads.	depends_on with condition service_healthy: the UI was started only after dl-api was Healthy (Figure 1, step 9).
Progress text labelled [stderr] in successful steps (4, 9, 27-29)	BuildKit and Compose write progress messages to stderr.	Exit code 0 in all of them; no action needed.
Build steps show CACHED	Layers were reused from earlier builds.	Build durations are not compared; sizes do not depend on them.
IDs in screenshots differ from the log	The screenshots were taken from a second run after the script had removed its containers (see Section 7).	Container and image IDs differ; sizes and results are identical.
Streamlit warnings in the manual UI test output	AppTest runs the UI script outside a Streamlit server ('bare mode').	Warnings are harmless; the scripted run filtered them (step 17).

5. Source Code

All files needed to build, run and test the application are listed. Lines longer than 100 characters are wrapped and continued with the symbol ↳. Line numbers refer to the real files.

5.1 docker-compose.yml

```
 1 # Full application: Flask API (model inside) + Streamlit UI, started together.
 2 name: dl-app
 3
 4 services:
 5   api:
 6     build: ./api
 7     image: dl-api:1.0
 8     container_name: dl-api
 9     ports:
10       - "8000:5000"          # host 8000 -> API port 5000 (host port 5000 is used by macOS AirPlay)
11     restart: unless-stopped
12
13   ui:
14     build: ./ui
15     image: dl-ui:1.0
16     container_name: dl-ui
17     environment:
18       API_URL: http://api:5000   # the UI reaches the API by its service name on the private network
19     ports:
20       - "8501:8501"
21     depends_on:
22       api:
23         condition: service_healthy   # start the UI only after the API reports "healthy"
24     restart: unless-stopped
25
26 networks:
27   default:
28     name: dl-app-net
```

5.2 api/Dockerfile

```
 1 # ---------- Stage 1: build (install dependencies into an isolated prefix) ----------
 2 FROM python:3.12-slim AS builder
 3 WORKDIR /build
 4 COPY requirements.txt .
 5 RUN pip install --no-cache-dir --prefix=/install -r requirements.txt
 6
 7 # ---------- Stage 2: runtime (only what is needed to serve the API) ----------
 8 FROM python:3.12-slim
 9
10 ENV PYTHONDONTWRITEBYTECODE=1 \
11     PYTHONUNBUFFERED=1 \
12     PORT=5000
13
14 # run as an unprivileged user instead of root
15 RUN useradd --create-home --uid 10001 --shell /usr/sbin/nologin appuser
16
17 WORKDIR /app
18 COPY --from=builder /install /usr/local
19 COPY app.py .
20 COPY model/ model/
21
22 USER appuser
23 EXPOSE 5000
24
25 HEALTHCHECK --interval=10s --timeout=3s --start-period=5s --retries=3 \
26   CMD ["python", "-c", "import urllib.request;
   ↳    urllib.request.urlopen('http://127.0.0.1:5000/health', timeout=2)"]
27
28 # production WSGI server (not the Flask development server)
29 CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "--access-logfile", "-", "app:app"]
```

5.3 ui/Dockerfile

```
# ---------- Stage 1: build (install dependencies into an isolated prefix) ----------
FROM python:3.12-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# ---------- Stage 2: runtime ----------
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    STREAMLIT_BROWSER_GATHER_USAGE_STATS=false \
    API_URL=http://api:5000

# run as an unprivileged user instead of root
RUN useradd --create-home --uid 10001 --shell /usr/sbin/nologin appuser

WORKDIR /app
COPY --from=builder /install /usr/local
COPY streamlit_app.py .

USER appuser
EXPOSE 8501

HEALTHCHECK --interval=10s --timeout=3s --start-period=10s --retries=3 \
  CMD ["python", "-c", "import urllib.request;
  ↳    urllib.request.urlopen('http://127.0.0.1:8501/_stcore/health', timeout=2)"]

CMD ["streamlit", "run", "streamlit_app.py", "--server.port=8501", "--server.address=0.0.0.0", "--
  ↳    server.headless=true"]
```

5.4 api/app.py - Flask API

```
"""
app.py - Flask deep-learning API (digit classifier).

Endpoints
  GET  /            service description and list of endpoints
  GET  /health      liveness check (used by the Docker HEALTHCHECK)
  GET  /model-info  model metadata (architecture, parameters, test accuracy)
  GET  /samples     one held-out example per digit (pixels + true label) for quick testing
  POST /predict     JSON {"pixels": [64 numbers]} or {"instances": [[64 numbers], ...]}

Run locally:   python app.py               (Flask development server)
Production:    gunicorn --bind 0.0.0.0:5000 app:app
"""
import json
import os
import time

import numpy as np
from flask import Flask, jsonify, request

BASE = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH = os.environ.get("MODEL_PATH", os.path.join(BASE, "model", "model.npz"))
MAX_BATCH = 100
N_FEATURES = 64

app = Flask(__name__)

_m = np.load(MODEL_PATH, allow_pickle=False)
W1, b1, W2, b2 = (_m[k].astype(np.float32) for k in ("W1", "b1", "W2", "b2"))
SAMPLES_X, SAMPLES_Y = _m["samples_x"], _m["samples_y"]
META = json.loads(str(_m["meta"]))
STARTED = time.time()
```

```

def predict_proba(x):
    """x: array (n, 64) with pixel values 0-16 -> softmax probabilities (n, 10)."""
    x = np.asarray(x, dtype=np.float32) / 16.0
    h = np.maximum(0.0, x @ W1 + b1)
    z = h @ W2 + b2
    e = np.exp(z - z.max(axis=1, keepdims=True))
    return e / e.sum(axis=1, keepdims=True)


def bad_request(message, status=400):
    return jsonify({"error": message}), status


@app.get("/")
def index():
    return jsonify({
        "service": "Flask deep-learning API (digit classifier)",
        "model": META["name"],
        "endpoints": {"GET /health": "liveness check", "GET /model-info": "model metadata",
                      "GET /samples": "example inputs with true labels",
                      "POST /predict": 'body: {"pixels": [64 numbers 0-16]} or {"instances": [[...],
    ...]}'},
    })


@app.get("/health")
def health():
    return jsonify({"status": "ok", "uptime_seconds": round(time.time() - STARTED, 1)})


@app.get("/model-info")
def model_info():
    return jsonify(META)


@app.get("/samples")
def samples():
    return jsonify([{"label": int(y), "pixels": [float(v) for v in x]} for x, y in zip(SAMPLES_X,
    SAMPLES_Y)])


@app.post("/predict")
def predict():
    data = request.get_json(silent=True)
    if not isinstance(data, dict):
        return bad_request('Request body must be a JSON object with "pixels" or "instances".')
    if "pixels" in data:
        rows = [data["pixels"]]
    elif "instances" in data:
        rows = data["instances"]
        if not isinstance(rows, list) or not rows:
            return bad_request('"instances" must be a non-empty list of 64-value lists.')
        if len(rows) > MAX_BATCH:
            return bad_request(f"At most {MAX_BATCH} instances per request.")
    else:
        return bad_request('Provide "pixels" (one image) or "instances" (list of images).')
    try:
        x = np.array(rows, dtype=np.float64)
    except (ValueError, TypeError):
        return bad_request("All pixel values must be numbers.")
    if x.ndim != 2 or x.shape[1] != N_FEATURES:
        return bad_request(f"Each image needs exactly {N_FEATURES} pixel values (flattened 8x8).")
    if not np.isfinite(x).all() or x.min() < 0 or x.max() > 16:
        return bad_request("Pixel values must be finite numbers between 0 and 16.")

    start = time.perf_counter()
    probs = predict_proba(x)
    elapsed = (time.perf_counter() - start) * 1000
    results = [{"digit": int(p.argmax()), "confidence": round(float(p.max()), 4),
                "probabilities": [round(float(v), 4) for v in p]} for p in probs]
    body = {"predictions": results, "count": len(results), "inference_ms": round(elapsed, 3)}
    if "pixels" in data:
        body.update(results[0])
    return jsonify(body)


@app.errorhandler(404)
def not_found(_):
```

```
    return jsonify({"error": "Route not found."}), 404


@app.errorhandler(405)
def not_allowed(_):
    return jsonify({"error": "Method not allowed for this route."}), 405


@app.errorhandler(500)
def server_error(_):
    return jsonify({"error": "Internal server error."}), 500


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=int(os.environ.get("PORT", 5000)))
```

5.5 ui/streamlit_app.py - Streamlit UI

```
"""
streamlit_app.py - Streamlit user interface for the deep-learning digit classifier.

The UI contains no model. It calls the Flask API (separate container) over HTTP:
    GET  /health, /model-info, /samples      POST /predict
The API address comes from the environment variable API_URL
(docker-compose sets API_URL=http://api:5000; default for local runs: http://localhost:5000).

Run locally:  API_URL=http://localhost:5000 streamlit run streamlit_app.py
"""
import os

import numpy as np
import requests
import streamlit as st

TIMEOUT = 5


def api_url():
    return os.environ.get("API_URL", "http://localhost:5000").rstrip("/")


def api_get(path):
    r = requests.get(api_url() + path, timeout=TIMEOUT)
    r.raise_for_status()
    return r.json()


def api_post(path, payload):
    r = requests.post(api_url() + path, json=payload, timeout=TIMEOUT)
    body = r.json()
    if r.status_code != 200:
        raise ValueError(body.get("error", f"HTTP {r.status_code}"))
    return body


def digit_image(pixels, scale=20):
    arr = np.array(pixels, dtype=float).reshape(8, 8) / 16.0 * 255.0
    return np.kron(arr, np.ones((scale, scale))).astype(np.uint8)


def show_result(result, true_label=None):
    c1, c2, c3 = st.columns(3)
    c1.metric("Predicted digit", str(result["digit"]))
    c2.metric("Confidence", f"{result['confidence'] * 100:.1f} %")
    if true_label is not None:
        c3.metric("True label", str(true_label))
```

```
        if result["digit"] == true_label:
            st.success("The prediction is correct.")
        else:
            st.warning("The prediction differs from the true label.")
    st.bar_chart({"probability": result["probabilities"]})
    st.caption(f"Inference time in the API: {result['inference_ms']} ms")


st.set_page_config(page_title="Digit Classifier", page_icon="", layout="wide")
st.title("Deep Learning Digit Classifier")
st.caption("Streamlit UI  ->  Flask API  ->  NumPy neural network  (UI and API run in separate Docker containers)")

# ---------------------------------------------------------------- sidebar: API status
with st.sidebar:
    st.header("API connection")
    st.write(f"Address: `{api_url()}`")
    try:
        health = api_get("/health")
        info = api_get("/model-info")
        st.success(f"API reachable (status: {health['status']})")
        st.metric("Model parameters", f"{info['parameters']:,}")
        st.metric("Model test accuracy", f"{info['test_accuracy'] * 100:.2f} %")
        st.caption(info["architecture"])
    except Exception as exc:  # the UI must not crash when the API is down
        st.error(f"API not reachable: {exc}")
        st.stop()

tab_sample, tab_pixels, tab_about = st.tabs(["Sample digits", "Enter pixel values", "About"])

# ---------------------------------------------------------------- tab 1: sample digits
with tab_sample:
    if "samples" not in st.session_state:
        st.session_state["samples"] = api_get("/samples")
    samples = {s["label"]: s["pixels"] for s in st.session_state["samples"]}
    digit = st.selectbox("Choose a held-out sample digit", list(range(10)), key="digit")
    left, right = st.columns([1, 3])
    left.image(digit_image(samples[digit]), caption=f"8x8 image of a {digit}", clamp=True)
    with right:
        if st.button("Predict", key="predict_sample", type="primary"):
            try:
                show_result(api_post("/predict", {"pixels": samples[digit]}), true_label=digit)
            except Exception as exc:
                st.error(f"Prediction failed: {exc}")

# ---------------------------------------------------------------- tab 2: pixel values
with tab_pixels:
    st.write("Enter 64 pixel values (0-16, flattened 8x8 image), separated by commas.")
    default = ", ".join(str(int(v)) for v in samples[8])
    text = st.text_area("Pixel values", value=default, height=120, key="pixels_text")
    if st.button("Predict pixel values", key="predict_text"):
        try:
            values = [float(v) for v in text.replace("\n", ",").split(",") if v.strip()]
            show_result(api_post("/predict", {"pixels": values}))
        except Exception as exc:
            st.error(f"Prediction failed: {exc}")

# ---------------------------------------------------------------- tab 3: about
with tab_about:
    st.markdown(
        "**Architecture**\n\n"
        "- `ui` container: this Streamlit application (port 8501)\n"
        "- `api` container: Flask + gunicorn serving the NumPy model (port 5000)\n"
        "- Both containers are started by Docker Compose and talk to each other over a private "
        "network using the service name `api`.\n\n"
        "**Model**: a neural network (64 -> 64 ReLU -> 10 softmax) trained on the UCI handwritten digits."
    )
```

5.6 tests/ui_apptest.py - UI integration test

```
"""
ui_apptest.py - integration test of the Streamlit UI against the REAL API.

It runs Streamlit's AppTest (the UI script executed in-process, driven like a user) and the UI
talks to the Flask API over HTTP. Inside the Docker setup it is executed in the ui container:
    docker compose exec -T ui python - < tests/ui_apptest.py
Environment: APP_PATH (default /app/streamlit_app.py), API_URL (set by the container).
"""
import json
import os
import sys

from streamlit.testing.v1 import AppTest

APP = os.environ.get("APP_PATH", "/app/streamlit_app.py")
good_api = os.environ.get("API_URL", "http://localhost:5000")
checks = []


def check(name, ok, detail):
    checks.append({"test": name, "passed": bool(ok), "detail": detail})
    print(f"[{'PASS' if ok else 'FAIL'}] {name}: {detail}")


def metric(at, label):
    return next((m.value for m in at.metric if m.label == label), None)


at = AppTest.from_file(APP, default_timeout=30).run()
check("UI page renders without an exception", not at.exception, f"exceptions={len(at.exception)}")
check("UI title is shown", any("Digit Classifier" in t.value for t in at.title), [t.value for t in
↳    at.title])
check("Sidebar reports that the API is reachable", any("API reachable" in s.value for s in
↳    at.sidebar.success),
      [s.value for s in at.sidebar.success])
check("Sidebar shows the model accuracy from the API", metric(at.sidebar, "Model test accuracy") is
↳    not None,
      metric(at.sidebar, "Model test accuracy"))

correct, rows = 0, []
for d in range(10):
    at.selectbox(key="digit").select(d).run()
    at.button(key="predict_sample").click().run()
    pred, conf = metric(at, "Predicted digit"), metric(at, "Confidence")
    ok = pred == str(d)
    correct += ok
    rows.append({"true": d, "predicted": pred, "confidence": conf})
    print(f"        digit {d} -> UI shows predicted {pred}, confidence {conf}")
check("UI classifies all 10 sample digits through the API", correct == 10, f"{correct}/10 correct")

sample = [s for s in at.session_state["samples"] if s["label"] == 3][0]["pixels"]
at.text_area(key="pixels_text").set_value(", ".join(str(int(v)) for v in sample)).run()
at.button(key="predict_text").click().run()
check("Pixel-value tab predicts a pasted image of the digit 3", metric(at, "Predicted digit") ==
↳    "3",
      f"predicted {metric(at, 'Predicted digit')}")

at.text_area(key="pixels_text").set_value("1, 2, 3").run()
at.button(key="predict_text").click().run()
errs = [e.value for e in at.error]
check("Invalid input shows the API error message in the UI",
      any("64 pixel values" in e for e in errs) and not at.exception, errs[:1])

os.environ["API_URL"] = "http://127.0.0.1:9"      # nothing listens here: simulate an API outage
down = AppTest.from_file(APP, default_timeout=30).run()
os.environ["API_URL"] = good_api
errs = [e.value for e in down.sidebar.error]
check("UI shows an error (no crash) when the API is unreachable", bool(errs) and not down.exception,
      (errs[0][:70] if errs else "no error shown"))

passed = sum(c["passed"] for c in checks)
print(f"\n{passed}/{len(checks)} UI integration checks passed")
sys.exit(0 if passed == len(checks) else 1)
```

5.7 tests/test_api.py - API tests

```
#!/usr/bin/env python3
"""
test_api.py - black-box tests for the running API (standard library only).

Usage:  python test_api.py --url http://localhost:5000 --out api_test_results.json
Works against a local server, gunicorn, or the Docker container (port 5000 published).
"""
import argparse
import json
import statistics
import sys
import time
import urllib.error
import urllib.request


def call(base, path, method="GET", body=None, raw=None, headers=None):
    data = raw if raw is not None else (json.dumps(body).encode() if body is not None else None)
    h = {"Content-Type": "application/json"} if data is not None else {}
    h.update(headers or {})
    req = urllib.request.Request(base + path, data=data, method=method, headers=h)
    t0 = time.perf_counter()
    try:
        with urllib.request.urlopen(req, timeout=10) as r:
            status, text = r.status, r.read().decode()
    except urllib.error.HTTPError as e:
        status, text = e.code, e.read().decode()
    ms = (time.perf_counter() - t0) * 1000
    try:
        parsed = json.loads(text)
    except ValueError:
        parsed = text
    return status, parsed, ms


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--url", default="http://localhost:5000")
    ap.add_argument("--out", default="api_test_results.json")
    a = ap.parse_args()
    base, checks = a.url.rstrip("/"), []

    def check(name, ok, detail):
        checks.append({"test": name, "passed": bool(ok), "detail": detail})
        print(f"[{'PASS' if ok else 'FAIL'}] {name}: {detail}")

    s, b, _ = call(base, "/health")
    check("GET /health returns 200 and status ok", s == 200 and b.get("status") == "ok", f"HTTP {s}
    ↳  {json.dumps(b)}")
    s, b, _ = call(base, "/")
    check("GET / lists the endpoints", s == 200 and "endpoints" in b, f"HTTP {s},
    ↳  endpoints={list(b.get('endpoints', {}))}")
    s, info, _ = call(base, "/model-info")
    check("GET /model-info returns model metadata", s == 200 and info.get("parameters") == 4810,
          f"HTTP {s}, parameters={info.get('parameters')},
    ↳  test_accuracy={info.get('test_accuracy')}")
    s, samples, _ = call(base, "/samples")
    check("GET /samples returns 10 labelled examples", s == 200 and len(samples) == 10, f"HTTP {s},
    ↳  {len(samples)} samples")

    correct, rows = 0, []
    for smp in samples:
        s, r, _ = call(base, "/predict", "POST", {"pixels": smp["pixels"]})
        ok = s == 200 and r.get("digit") == smp["label"]
```

```
        correct += ok
        rows.append({"true": smp["label"], "predicted": r.get("digit"), "confidence":
    r.get("confidence")})
    check("POST /predict classifies the 10 samples", correct == 10, f"{correct}/10 correct")
    for row in rows: print(f"        true {row['true']} -> predicted {row['predicted']} (confidence
    {row['confidence']})")

    s, r, _ = call(base, "/predict", "POST", {"instances": [x["pixels"] for x in samples]})
    batch_ok = s == 200 and r.get("count") == 10 and [p["digit"] for p in r["predictions"]] ==
    [x["label"] for x in samples]
    check("POST /predict batch of 10 instances", batch_ok, f"HTTP {s}, count={r.get('count')},
    inference_ms={r.get('inference_ms')}")

    s, r, _ = call(base, "/predict", "POST", {"pixels": [1, 2, 3]})
    check("Wrong input length is rejected with 400", s == 400, f"HTTP {s} {json.dumps(r)}")
    s, r, _ = call(base, "/predict", "POST", {"pixels": ["a"] * 64})
    check("Non-numeric pixels are rejected with 400", s == 400, f"HTTP {s} {json.dumps(r)}")
    s, r, _ = call(base, "/predict", "POST", {"pixels": [99] * 64})
    check("Out-of-range pixels are rejected with 400", s == 400, f"HTTP {s} {json.dumps(r)}")
    s, r, _ = call(base, "/predict", "POST", raw=b"not json")
    check("Invalid JSON is rejected with 400", s == 400, f"HTTP {s} {json.dumps(r)}")
    s, r, _ = call(base, "/nope")
    check("Unknown route returns JSON 404", s == 404 and isinstance(r, dict), f"HTTP {s}
    {json.dumps(r)}")
    s, r, _ = call(base, "/predict")
    check("GET on /predict returns JSON 405", s == 405 and isinstance(r, dict), f"HTTP {s}
    {json.dumps(r)}")

    lat = [call(base, "/predict", "POST", {"pixels": samples[i % 10]["pixels"]})[2] for i in
    range(50)]
    summary = {"requests": 50, "mean_ms": round(statistics.mean(lat), 2), "median_ms":
    round(statistics.median(lat), 2),
               "max_ms": round(max(lat), 2)}
    check("Latency of 50 sequential /predict calls measured", True, json.dumps(summary))

    passed = sum(c["passed"] for c in checks)
    print(f"\n{passed}/{len(checks)} checks passed")
    json.dump({"base_url": base, "passed": passed, "total": len(checks), "checks": checks,
    "predictions": rows,
               "latency": summary}, open(a.out, "w"), indent=2)
    sys.exit(0 if passed == len(checks) else 1)


if __name__ == "__main__":
    main()
```

5.8 api/train_model.py - model training

```
#!/usr/bin/env python3
"""
train_model.py - trains the deep-learning model served by the Flask API.

A small fully connected neural network (64 -> 64 ReLU -> 10 softmax) is trained from
scratch in NumPy on the UCI handwritten-digits dataset (8x8 images, 1,797 samples) and
exported to model/model.npz. The API only needs NumPy at inference time, which keeps the
Docker image small. scikit-learn is used here only to load the dataset (training-time
dependency, listed in requirements-train.txt and NOT installed in the image).

Usage:  python train_model.py
"""
import json
import os
from datetime import date

```

```
import numpy as np
from sklearn.datasets import load_digits

SEED, EPOCHS, BATCH, LR, HIDDEN = 42, 80, 32, 0.002, 64
OUT = os.path.join(os.path.dirname(os.path.abspath(__file__)), "model", "model.npz")


def softmax(z):
    e = np.exp(z - z.max(axis=1, keepdims=True))
    return e / e.sum(axis=1, keepdims=True)


def forward(x, p):
    h = np.maximum(0, x @ p["W1"] + p["b1"])
    return h, softmax(h @ p["W2"] + p["b2"])


def main():
    d = load_digits()
    X, y = d.data.astype(np.float64) / 16.0, d.target
    rng = np.random.default_rng(SEED)
    idx = rng.permutation(len(X))
    cut = int(len(X) * 0.8)
    tr, te = idx[:cut], idx[cut:]
    Xtr, ytr, Xte, yte = X[tr], y[tr], X[te], y[te]

    p = {"W1": rng.normal(0, np.sqrt(2 / 64), (64, HIDDEN)), "b1": np.zeros(HIDDEN),
         "W2": rng.normal(0, np.sqrt(2 / HIDDEN), (HIDDEN, 10)), "b2": np.zeros(10)}
    m = {k: np.zeros_like(v) for k, v in p.items()}
    v = {k: np.zeros_like(val) for k, val in p.items()}
    t = 0
    for ep in range(1, EPOCHS + 1):
        order = rng.permutation(len(Xtr))
        for i in range(0, len(order), BATCH):
            b = order[i:i + BATCH]
            xb, yb = Xtr[b], ytr[b]
            h, pr = forward(xb, p)
            dz2 = pr.copy(); dz2[np.arange(len(b)), yb] -= 1; dz2 /= len(b)
            g = {"W2": h.T @ dz2, "b2": dz2.sum(0)}
            dh = dz2 @ p["W2"].T; dh[h <= 0] = 0
            g["W1"], g["b1"] = xb.T @ dh, dh.sum(0)
            t += 1
            for k in p:  # Adam update
                m[k] = 0.9 * m[k] + 0.1 * g[k]
                v[k] = 0.999 * v[k] + 0.001 * g[k] ** 2
                p[k] -= LR * (m[k] / (1 - 0.9 ** t)) / (np.sqrt(v[k] / (1 - 0.999 ** t)) + 1e-8)
        if ep % 20 == 0 or ep == 1:
            tra = (forward(Xtr, p)[1].argmax(1) == ytr).mean()
            tea = (forward(Xte, p)[1].argmax(1) == yte).mean()
            print(f"epoch {ep:3d}  train acc {tra:.4f}  test acc {tea:.4f}")

    train_acc = float((forward(Xtr, p)[1].argmax(1) == ytr).mean())
    test_acc = float((forward(Xte, p)[1].argmax(1) == yte).mean())
    params = int(sum(v_.size for v_ in p.values()))

    # one held-out test example per digit, served by GET /samples for easy verification
    sx, sy = [], []
    for digit in range(10):
        j = np.where(yte == digit)[0][0]
        sx.append(d.data[te[j]]); sy.append(digit)

    meta = {"name": "digits-mlp", "dataset": "UCI handwritten digits (sklearn load_digits), 8x8 grayscale",
            "architecture": "Input 64 -> Dense 64 + ReLU -> Dense 10 + Softmax", "parameters": params,
            "input": "64 pixel values in the range 0-16 (flattened 8x8 image)", "classes": list(range(10)),
            "train_samples": int(len(tr)), "test_samples": int(len(te)),
            "train_accuracy": round(train_acc, 4), "test_accuracy": round(test_acc, 4),
            "epochs": EPOCHS, "seed": SEED, "trained_on": str(date.today()), "numpy_version": np.__version__}
    os.makedirs(os.path.dirname(OUT), exist_ok=True)
    np.savez(OUT, W1=p["W1"].astype(np.float32), b1=p["b1"].astype(np.float32),
             W2=p["W2"].astype(np.float32), b2=p["b2"].astype(np.float32),
             samples_x=np.array(sx, dtype=np.float32), samples_y=np.array(sy, dtype=np.int64),
             meta=np.array(json.dumps(meta)))
    print(json.dumps(meta, indent=2))
    print("saved", OUT, f"({os.path.getsize(OUT) / 1024:.1f} KB)")


```

```
93 if __name__ == "__main__":
94     main()
```

5.9 api/requirements.txt

```
 1 blinker==1.9.0
 2 click==8.5.0
 3 Flask==3.1.3
 4 gunicorn==26.2.0
 5 itsdangerous==2.2.0
 6 Jinja2==3.1.6
 7 MarkupSafe==3.0.4
 8 numpy==2.5.3
 9 Werkzeug==3.1.9
```

5.10 ui/requirements.txt

```
 1 altair==6.3.0
 2 anyio==4.15.1
 3 attrs==26.1.0
 4 certifi==2026.7.22
 5 charset-normalizer==3.5.2
 6 click==8.5.0
 7 h11==0.16.0
 8 httptools==0.9.0
 9 idna==3.20
10 itsdangerous==2.2.0
11 Jinja2==3.1.6
12 jsonschema-specifications==2025.9.1
13 jsonschema==4.26.0
14 MarkupSafe==3.0.4
15 narwhals==2.26.0
16 numpy==2.5.3
17 packaging==26.3
18 pandas==3.0.6
19 pillow==12.3.0
20 protobuf==7.36.2
21 pyarrow==25.0.1
22 pydeck==0.9.3
23 python-dateutil==2.9.0.post0
24 python-multipart==0.0.32
25 referencing==0.37.0
26 requests==2.34.2
27 rpds-py==2026.9.1
28 six==1.17.0
29 starlette==1.7.0
30 streamlit==1.65.0
31 toml==0.10.2
32 typing_extensions==4.16.0
33 urllib3==2.8.0
34 uvicorn==0.54.0
35 watchdog==6.0.0
36 websockets==17.2
```

5.11 api/.dockerignore

```
 1 # keep the build context (and therefore the image) small
 2 .git
 3 __pycache__/
 4 *.pyc
 5 venv/
 6 .venv/
 7 *.md
 8 *.txt
 9 !requirements.txt
10 train_model.py
11 Dockerfile*
12 .dockerignore
```

5.12 ui/.dockerignore

```
.git
__pycache__/
*.pyc
venv/
.venv/
*.md
Dockerfile*
.dockerignore
```

5.13 task11_docker.py - Docker workflow runner

```
#!/usr/bin/env python3
"""
task11_docker.py - runs the complete Docker workflow for Task 11 (Full Application
Containerization: model + Flask API + Streamlit UI) and records the REAL output of every command.

Steps: verify Docker/Compose -> validate config and build both images -> start the application with
docker compose -> test API, UI and the container integration -> inspect containers and network ->
resilience checks (restart / stop / start the API) -> clean up.

Usage:
    1. Start Docker Desktop and wait for "Engine running".
    2. Run from the project folder (the one containing docker-compose.yml):
           python3 task11_docker.py
Outputs:
    docker_command_log.txt      plain-text log of every command, output and exit code
    docker_results.json         structured results (used to build the report)
    api_test_results.json       results of tests/test_api.py against the API container
    ui_integration_test.txt     output of the UI integration test run inside the ui container
    image_size_summary.txt      sizes of the two images
Host ports used: 8000 (API) and 8501 (Streamlit UI). They must be free.
"""
import json
import os
import socket
import subprocess
import sys
from datetime import datetime

HERE = os.path.dirname(os.path.abspath(__file__))
os.chdir(HERE)

TABLE = "table {{.Names}}\\t{{.Image}}\\t{{.Status}}\\t{{.Ports}}"


def wait(name, tries=60):
    return ("for i in $(seq 1 %d); do s=$(docker inspect --format '{{.State.Health.Status}}' %s); "
            "echo \"check $i: $s\"; [ \"$s\" = healthy ] && break; sleep 2; done; [ \"$s\" = healthy
    ]" % (tries, name))


PY_HEALTH = "\"import urllib.request;
    print(urllib.request.urlopen('http://api:5000/health').read().decode())\""
PY_MODS = ("\"import importlib.util as u; print('flask installed:', u.find_spec('flask') is not
    None); "
           "print('streamlit installed:', u.find_spec('streamlit') is not None); print('numpy
    installed:', u.find_spec('numpy') is not None)\"")

A = "1. Verify Docker and Compose"
B = "2. Validate the configuration and build the images"
C = "3. Start the full application with Docker Compose"
D = "4. Test the application"
E = "5. Inspect the containers and the network"
F = "6. Resilience and dependency checks"
G = "7. Cleanup"

# (section, title, command, critical)
STEPS = [
    (A, "Docker version", "docker --version && docker version --format 'Client: {{.Client.Version}}
    | Server: {{.Server.Version}}'", True),
    (A, "Docker Compose version", "docker compose version", True),
    (B, "Services defined in docker-compose.yml", "docker compose config --services", True),
    (B, "Build both images (api and ui)", "docker compose build", True),
    (B, "List the images", "docker image ls --filter 'reference=dl-*' --format 'table
    {{.Repository}}\\t{{.Tag}}\\t{{.ID}}\\t{{.Size}}'", False),
```

```
    (B, "Size of the API image in bytes", "docker image inspect dl-api:1.0 --format '{{.Size}}'",
    ↳  False),
    (B, "Size of the UI image in bytes", "docker image inspect dl-ui:1.0 --format '{{.Size}}'",
    ↳  False),
    (B, "Layer history of the UI image", "docker history dl-ui:1.0", False),
    (C, "Start both containers (docker compose up -d)", "docker compose up -d", True),
    (C, "Wait until the API container is healthy", wait("dl-api"), True),
    (C, "Wait until the UI container is healthy", wait("dl-ui"), True),
    (C, "List the running containers", "docker ps --filter name=dl- --format '" + TABLE + "'",
    ↳  False),
    (D, "API test suite (tests/test_api.py)", "python3 tests/test_api.py --url http://localhost:8000
    ↳   --out api_test_results.json", False),
    (D, "UI health endpoint", "curl -s http://localhost:8501/_stcore/health", False),
    (D, "UI main page", "curl -s -o /dev/null -w 'HTTP status: %{http_code}\\n'
    ↳  http://localhost:8501/", False),
    (D, "UI container reaches the API container by service name", "docker compose exec -T ui python
    ↳  -c " + PY_HEALTH, False),
    (D, "UI integration test inside the ui container (Streamlit AppTest against the real API)",
    ↳  "docker compose exec -T ui python - < tests/ui_apptest.py 2>&1 | grep -v -i
    ↳  'warning\\|ScriptRunContext'; exit ${PIPESTATUS[0]}", False),
    (E, "User inside both containers (should not be root)", "docker exec dl-api id && docker exec
    ↳  dl-ui id", False),
    (E, "Container configuration and health", "docker inspect dl-api dl-ui --format '{{.Name}}:
    ↳  User={{.Config.User}} | Health={{.State.Health.Status}} | Image={{.Config.Image}} |
    ↳  Restart={{.HostConfig.RestartPolicy.Name}}'", False),
    (E, "Containers attached to the private network", "docker network inspect dl-app-net --format
    ↳  '{{range .Containers}}{{.Name}} -> {{.IPv4Address}}{{println}}{{end}}'", False),
    (E, "API address configured in the UI container", "docker exec dl-ui printenv API_URL", False),
    (E, "Installed components: api container", "docker exec dl-api python -c " + PY_MODS, False),
    (E, "Installed components: ui container", "docker exec dl-ui python -c " + PY_MODS, False),
    (E, "Container logs (last lines of both services)", "docker compose logs --tail 6", False),
    (E, "Resource usage of both containers", "docker stats --no-stream dl-api dl-ui", False),
    (F, "Restart the API container; the UI can reach it again", "docker restart dl-api && " +
    ↳  wait("dl-api") + " && docker compose exec -T ui python -c " + PY_HEALTH, False),
    (F, "Stop the API; the UI container keeps running", "docker compose stop api && sleep 2 && curl
    ↳  -s http://localhost:8501/_stcore/health && echo && docker ps -a --filter name=dl- --format
    ↳  'table {{.Names}}\\t{{.Status}}'", False),
    (F, "Start the API again and repeat the UI integration test", "docker compose start api && " +
    ↳  wait("dl-api") + " && docker compose exec -T ui python - < tests/ui_apptest.py 2>&1 | grep -v -i
    ↳   'warning\\|ScriptRunContext' | tail -4", False),
    (G, "Stop and remove the containers and the network (images are kept)", "docker compose down &&
    ↳  docker ps -a --format 'table {{.Names}}\\t{{.Status}}'", False),
]


def run_cmd(cmd, timeout=2400):
    t0 = datetime.now()
    try:
        p = subprocess.run(["bash", "-c", cmd], capture_output=True, text=True, timeout=timeout)
        out = (p.stdout or "") + (("\n[stderr]\n" + p.stderr) if p.stderr.strip() else "")
        rc = p.returncode
    except subprocess.TimeoutExpired:
        out, rc = "Command timed out.", 124
    return {"command": cmd, "output": out.rstrip(), "returncode": rc, "time": t0.strftime("%Y-%m-%d
    ↳  %H:%M:%S")}


def port_in_use(port):
    with socket.socket() as s:
        s.settimeout(1)
        return s.connect_ex(("127.0.0.1", port)) == 0


def save(results):
    json.dump(results, open("docker_results.json", "w"), indent=2)
    with open("docker_command_log.txt", "w") as f:
        f.write(f"Task 11 Docker command log - generated {datetime.now():%Y-%m-%d %H:%M:%S}\n" + "="
    ↳    * 78 + "\n")
        for i, r in enumerate(results, 1):
            f.write(f"\n[{i}] {r['section']} - {r['title']}  ({r['time']})\n$
    ↳  {r['command']}\n{r['output']}\n"
                    f"(exit code {r['returncode']})\n" + "-" * 78 + "\n")


def main():
    if run_cmd("docker info", 60)["returncode"] != 0:
        sys.exit("Docker is not running. Start Docker Desktop, wait for 'Engine running', then run
    ↳  again.")
    for f in ("docker-compose.yml", "api/Dockerfile", "ui/Dockerfile", "api/app.py",
    ↳  "ui/streamlit_app.py",
```

```
             "api/model/model.npz", "tests/test_api.py", "tests/ui_apptest.py"):
        if not os.path.exists(f):
            sys.exit(f"Missing file: {f}. Run this script from the project folder.")
    # remove leftovers of an earlier run so that names and ports are free
    run_cmd("docker compose down --remove-orphans; docker rm -f dl-api dl-ui; true")
    busy = [p for p in (8000, 8501) if port_in_use(p)]
    if busy:
        sys.exit(f"Host port(s) {busy} are already in use by another program or container. "
                 f"Stop it (see 'docker ps') and run this script again.")

    results = []
    for i, (section, title, cmd, critical) in enumerate(STEPS, 1):
        r = run_cmd(cmd)
        r.update({"section": section, "title": title})
        results.append(r)
        print(f"\n### [{i}/{len(STEPS)}] {title}\n$ {cmd}\n{r['output'][-3000:]}\n(exit code
    ↳  {r['returncode']})")
        if critical and r["returncode"] != 0:
            save(results)
            print(f"\nSTOPPED: the critical step '{title}' failed. Nothing after it can work.\n"
                  f"Please send me the file docker_command_log.txt (it contains the error).")
            run_cmd("docker compose down --remove-orphans; true")
            sys.exit(1)

    def out_of(title):
        return next(x for x in results if x["title"] == title)["output"]

    def num(title):
        d = "".join(ch for ch in out_of(title).split("\n")[0] if ch.isdigit())
        return int(d) if d else None
    a, u = num("Size of the API image in bytes"), num("Size of the UI image in bytes")
    if a and u:
        txt = (f"dl-api:1.0 (Flask + gunicorn + NumPy model) : {a:>13,} bytes = {a / 1e6:8.1f} MB\n"
               f"dl-ui:1.0  (Streamlit UI)                  : {u:>13,} bytes = {u / 1e6:8.1f}
    ↳  MB\n")
        open("image_size_summary.txt", "w").write(txt)
        print("\n" + txt)
    open("ui_integration_test.txt", "w").write(
        out_of("UI integration test inside the ui container (Streamlit AppTest against the real
    ↳  API)") + "\n")
    save(results)
    ok = sum(1 for r in results if r["returncode"] == 0)
    print(f"\nDone: {ok}/{len(results)} commands succeeded. Files: docker_command_log.txt,
    ↳  docker_results.json, "
          f"api_test_results.json, ui_integration_test.txt, image_size_summary.txt")


if __name__ == "__main__":
    main()
```

6. Command Execution Log

All 29 commands below were run by `task11_docker.py`. The command, output and exit code are copied from `docker_results.json`; the observation under each step explains what the output shows. The long build output is shortened (marked with '...'); the complete text is in `docker_command_log.txt`.

6.1 Verify Docker and Compose (Steps 1-2)

Concept: Docker Compose describes a multi-container application in one file (docker-compose.yml). The command 'docker compose' builds the images, creates the network and starts the containers together.

Step 1: Docker version (exit code 0)

```
$ docker --version && docker version --format 'Client: {{.Client.Version}} | Server: {{.Server.Version}}'
```

```
Docker version 29.8.2, build 7fc2dff
Client: 29.8.2 | Server: 29.8.2
```

Observation: Client and server both report version 29.8.2, so the CLI is talking to the Docker Desktop engine correctly.

Step 2: Docker Compose version (exit code 0)

```
$ docker compose version
```

```
Docker Compose version v5.5.1
```

Observation: Docker Compose v5.5.1 is available as the plugin 'docker compose'.

6.2 Validate the Configuration and Build the Images (Steps 3-8)

Concept: Each service has its own Dockerfile and its own pinned requirements.txt, so the API image contains Flask and the model, and the UI image contains Streamlit. Multi-stage builds keep build leftovers out of both images.

Step 3: Services defined in docker-compose.yml (exit code 0)

```
$ docker compose config --services
```

```
api
ui
```

Observation: Compose reads docker-compose.yml, validates it and lists the two services of the application: api and ui.

Step 4: Build both images (api and ui) (exit code 0)

```
$ docker compose build
```

```
#1 [internal] load local bake definitions
#1 reading from stdin 1.04kB done
#1 DONE 0.0s

#2 [api internal] load build definition from Dockerfile
#2 transferring dockerfile: 1.02kB done
#2 DONE 0.0s

... [219 lines omitted - full output in docker_command_log.txt]
#27 exporting manifest list sha256:a213b7a307747019eae5b43ac8ea3ab87d734ce89d8373f3f46ca2f78ad3edfe done
#27 naming to docker.io/library/dl-ui:1.0 done
#27 unpacking to docker.io/library/dl-ui:1.0
#27 unpacking to docker.io/library/dl-ui:1.0 2.4s done
#27 DONE 16.1s
#28 [ui] resolving provenance for metadata file
#28 DONE 0.0s

[stderr]
 Image dl-ui:1.0 Building
 Image dl-api:1.0 Building
 Image dl-api:1.0 Built
 Image dl-ui:1.0 Built
```

Observation: One command builds both images. The API image reuses cached layers (every API layer is CACHED, the Dockerfile is the one of an earlier build). The UI image is built from its Dockerfile: pip installed the 36 pinned packages (38.7 s; for example numpy 15.7 MB, pandas 10.3 MB, pyarrow 46.8 MB, streamlit 10.3 MB), the install layer was copied (1.4 s) and the image exported (16.1 s). The images are named dl-api:1.0 and dl-ui:1.0. BuildKit writes its progress to stderr; the exit code is 0.

Step 5: List the images (exit code 0)

```
$ docker image ls --filter 'reference=dl-*' --format 'table {{.Repository}}\t{{.Tag}}\t{{.ID}}\t{{.Size}}'
```

```
REPOSITORY          TAG       IMAGE ID       SIZE
dl-ui               1.0       a213b7a30774   779MB
dl-api              1.0       d081e9baa059   303MB
dl-deploy-api       latest    1d9646e0048f   2.28GB
dl-deploy-frontend  latest    6371e70a23bc   792MB
dl-digits           1.0       3c11f996f9d5   603MB
dl-app              1.0       34d8650d4c16   324MB
```

Observation: Two images were built: dl-ui:1.0 (779 MB) and dl-api:1.0 (303 MB). The other dl-* images in the list belong to earlier exercises.

Step 6: Size of the API image in bytes (exit code 0)

```
$ docker image inspect dl-api:1.0 --format '{{.Size}}'
```

```
303293481
```

Observation: The API image occupies 303,293,481 bytes (about 303.3 MB).

Step 7: Size of the UI image in bytes (exit code 0)

```
$ docker image inspect dl-ui:1.0 --format '{{.Size}}'
```

```
778900479
```

Observation: The UI image occupies 778,900,479 bytes (about 778.9 MB), about 2.6 times the API image, because Streamlit brings pandas, pyarrow, NumPy, Pillow and many other packages.

Step 8: Layer history of the UI image (exit code 0)

```
$ docker history dl-ui:1.0
```

```
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
a213b7a30774   17 seconds ago   CMD ["streamlit" "run" "streamlit_app.py" "-…   0B        buildkit.dockerfile.v0
<missing>      17 seconds ago   HEALTHCHECK {Test:[CMD python -c import urll…   0B        buildkit.dockerfile.v0
<missing>      17 seconds ago   EXPOSE [8501/tcp]                               0B        buildkit.dockerfile.v0
<missing>      17 seconds ago   USER appuser                                    0B        buildkit.dockerfile.v0
<missing>      17 seconds ago   COPY streamlit_app.py . # buildkit              16.4kB    buildkit.dockerfile.v0
<missing>      17 seconds ago   COPY /install /usr/local # buildkit             448MB     buildkit.dockerfile.v0
<missing>      57 seconds ago   WORKDIR /app                                    8.19kB    buildkit.dockerfile.v0
<missing>      58 seconds ago   RUN /bin/sh -c useradd --create-home --uid 1…   69.6kB    buildkit.dockerfile.v0
<missing>      58 seconds ago   ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFER…   0B        buildkit.dockerfile.v0
<missing>      4 days ago       CMD ["python3"]                                 0B        buildkit.dockerfile.v0
<missing>      4 days ago       RUN /bin/sh -c set -eux;  for src in idle3 p…   16.4kB    buildkit.dockerfile.v0
<missing>      4 days ago       RUN /bin/sh -c set -eux;   savedAptMark="$(a…   44.6MB    buildkit.dockerfile.v0
<missing>      4 days ago       ENV PYTHON_SHA256=c2c4321961fab0fb999d66e0ce…   0B        buildkit.dockerfile.v0
<missing>      4 days ago       ENV PYTHON_VERSION=3.12.15                      0B        buildkit.dockerfile.v0
<missing>      4 days ago       ENV GPG_KEY=7169605F62C751356D054A26A821E680…   0B        buildkit.dockerfile.v0
<missing>      4 days ago       RUN /bin/sh -c set -eux;  apt-get update;  a…   4.99MB    buildkit.dockerfile.v0
<missing>      4 days ago       ENV LANG=C.UTF-8                                0B        buildkit.dockerfile.v0
<missing>      4 days ago       ENV PATH=/usr/local/bin:/usr/local/sbin:/usr…   0B        buildkit.dockerfile.v0
<missing>      5 days ago       # debian.sh --arch 'arm64' out/ 'trixie' '@1…   110MB     debuerreotype 0.17
```

Observation: The history of the UI image shows the base layers of python:3.12-slim (110 MB Debian arm64 layer, 44.6 MB Python layer, 4.99 MB apt layer, Python 3.12.15), the user layer (69.6 kB), the installed packages copied from the builder stage (448 MB) and streamlit_app.py (16.4 kB). USER, EXPOSE, HEALTHCHECK and CMD only set metadata (0 B).

6.3 Start the Full Application with Docker Compose (Steps 9-12)

Concept: Compose starts the services in the order given by depends_on. With condition service_healthy the UI is started only after the API container reports (healthy) through its HEALTHCHECK.

Step 9: Start both containers (docker compose up -d) (exit code 0)

```
$ docker compose up -d
```

```
[stderr]
 Network dl-app-net Creating
 Network dl-app-net Creating
 Network dl-app-net Created
 Network dl-app-net Created
 Container dl-api Creating
 Container dl-api Created
 Container dl-ui Creating
 Container dl-ui Created
 Container dl-api Starting
 Container dl-api Started
 Container dl-api Waiting
 Container dl-api Healthy
 Container dl-ui Starting
 Container dl-ui Started
```

Observation: docker compose up -d created the network dl-app-net and both containers, started dl-api, waited until it was Healthy and only then started dl-ui. This is the effect of depends_on with condition service_healthy.

Step 10: Wait until the API container is healthy (exit code 0)

```
$ for i in $(seq 1 60); do s=$(docker inspect --format '{{.State.Health.Status}}' dl-api); echo "check $i: $s";
  [ "$s" = healthy ] && break; sleep 2; done; [ "$s" = healthy ]
```

```
check 1: healthy
```

Observation: The API container was already healthy at the first check, because Compose had waited for it before starting the UI.

Step 11: Wait until the UI container is healthy (exit code 0)

```
$ for i in $(seq 1 60); do s=$(docker inspect --format '{{.State.Health.Status}}' dl-ui); echo "check $i: $s"; [
   "$s" = healthy ] && break; sleep 2; done; [ "$s" = healthy ]
```

```
check 1: starting
check 2: starting
check 3: starting
check 4: healthy
```

Observation: The UI container passed through starting (checks 1 to 3) and became healthy at check 4, about 6 seconds after it was started.

Step 12: List the running containers (exit code 0)

```
$ docker ps --filter name=dl- --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}'
```

```
NAMES     IMAGE        STATUS                    PORTS
dl-ui     dl-ui:1.0    Up 6 seconds (healthy)    0.0.0.0:8501->8501/tcp, [::]:8501->8501/tcp
dl-api    dl-api:1.0   Up 12 seconds (healthy)   0.0.0.0:8000->5000/tcp, [::]:8000->5000/tcp
```

Observation: Both containers are healthy: dl-ui (Up 6 seconds) with port 8501->8501 and dl-api (Up 12 seconds) with host port 8000 mapped to container port 5000.

6.4 Test the Application (Steps 13-17)

Concept: The application is tested on three levels: the API from outside (published port 8000), the UI (health endpoint and page) and the integration of both containers (UI to API over the private network, including the UI test that runs inside the ui container).

Step 13: API test suite (tests/test_api.py) (exit code 0)

```
$ python3 tests/test_api.py --url http://localhost:8000 --out api_test_results.json
```

```
[PASS] GET /health returns 200 and status ok: HTTP 200 {"status": "ok", "uptime_seconds": 11.6}
[PASS] GET / lists the endpoints: HTTP 200, endpoints=['GET /health', 'GET /model-info', 'GET /samples', 'POST
  /predict']
[PASS] GET /model-info returns model metadata: HTTP 200, parameters=4810, test_accuracy=0.9722
[PASS] GET /samples returns 10 labelled examples: HTTP 200, 10 samples
[PASS] POST /predict classifies the 10 samples: 10/10 correct
        true 0 -> predicted 0 (confidence 1.0)
        true 1 -> predicted 1 (confidence 0.9989)
        true 2 -> predicted 2 (confidence 1.0)
        true 3 -> predicted 3 (confidence 1.0)
        true 4 -> predicted 4 (confidence 1.0)
        true 5 -> predicted 5 (confidence 1.0)
        true 6 -> predicted 6 (confidence 0.9972)
        true 7 -> predicted 7 (confidence 1.0)
        true 8 -> predicted 8 (confidence 0.9951)
        true 9 -> predicted 9 (confidence 0.9837)
[PASS] POST /predict batch of 10 instances: HTTP 200, count=10, inference_ms=0.666
[PASS] Wrong input length is rejected with 400: HTTP 400 {"error": "Each image needs exactly 64 pixel values
  (flattened 8x8)."}
[PASS] Non-numeric pixels are rejected with 400: HTTP 400 {"error": "All pixel values must be numbers."}
[PASS] Out-of-range pixels are rejected with 400: HTTP 400 {"error": "Pixel values must be finite numbers
  between 0 and 16."}
[PASS] Invalid JSON is rejected with 400: HTTP 400 {"error": "Request body must be a JSON object with \"pixels\"
   or \"instances\"."}
[PASS] Unknown route returns JSON 404: HTTP 404 {"error": "Route not found."}
[PASS] GET on /predict returns JSON 405: HTTP 405 {"error": "Method not allowed for this route."}
[PASS] Latency of 50 sequential /predict calls measured: {"requests": 50, "mean_ms": 1.56, "median_ms": 1.51,
  "max_ms": 2.51}
13/13 checks passed
```

Observation: All 13 API checks passed: the four GET routes, 10 of 10 sample digits, a batch of 10 instances (0.666 ms inference), four kinds of invalid input answered with 400, JSON 404 and 405 errors and the latency of

50 sequential requests (mean 1.56 ms, median 1.51 ms, maximum 2.51 ms).

Step 14: UI health endpoint (exit code 0)

```
$ curl -s http://localhost:8501/_stcore/health
```

```
ok
```

Observation: Streamlit's health endpoint /_stcore/health answers ok, which is also what the HEALTHCHECK of the ui container calls.

Step 15: UI main page (exit code 0)

```
$ curl -s -o /dev/null -w 'HTTP status: %{http_code}\n' http://localhost:8501/
```

```
HTTP status: 200
```

Observation: The Streamlit main page is served with HTTP status 200 on host port 8501.

Step 16: UI container reaches the API container by service name (exit code 0)

```
$ docker compose exec -T ui python -c "import urllib.request;
  print(urllib.request.urlopen('http://api:5000/health').read().decode())"
```

```
{"status":"ok","uptime_seconds":12.2}
```

Observation: From inside the ui container the name api resolves and the API answers {"status":"ok"}. This proves that the two containers communicate over the private network by service name.

Step 17: UI integration test inside the ui container (Streamlit AppTest against the real API) (exit code 0)

```
$ docker compose exec -T ui python - < tests/ui_apptest.py 2>&1 | grep -v -i 'warning\|ScriptRunContext'; exit
  ${PIPESTATUS[0]}
```

```
[PASS] UI page renders without an exception: exceptions=0
[PASS] UI title is shown: ['Deep Learning Digit Classifier']
[PASS] Sidebar reports that the API is reachable: ['API reachable (status: ok)']
[PASS] Sidebar shows the model accuracy from the API: 97.22 %
        digit 0 -> UI shows predicted 0, confidence 100.0 %
        digit 1 -> UI shows predicted 1, confidence 99.9 %
        digit 2 -> UI shows predicted 2, confidence 100.0 %
        digit 3 -> UI shows predicted 3, confidence 100.0 %
        digit 4 -> UI shows predicted 4, confidence 100.0 %
        digit 5 -> UI shows predicted 5, confidence 100.0 %
        digit 6 -> UI shows predicted 6, confidence 99.7 %
        digit 7 -> UI shows predicted 7, confidence 100.0 %
        digit 8 -> UI shows predicted 8, confidence 99.5 %
        digit 9 -> UI shows predicted 9, confidence 98.4 %
[PASS] UI classifies all 10 sample digits through the API: 10/10 correct
[PASS] Pixel-value tab predicts a pasted image of the digit 3: predicted 3
[PASS] Invalid input shows the API error message in the UI: ['Prediction failed: Each image needs exactly 64
  pixel values (flattened 8x8).']
[PASS] UI shows an error (no crash) when the API is unreachable: API not reachable:
  HTTPConnectionPool(host='127.0.0.1', port=9): Max r

8/8 UI integration checks passed
```

Observation: All 8 UI integration checks passed inside the ui container, with the UI talking to the real API container: the page renders, the title and the API status are shown, the sidebar shows the model accuracy of 97.22 %, all 10 sample digits are classified correctly through the UI, a pasted pixel list is classified, the API's error message for invalid input is shown, and an unreachable API produces an error message instead of a crash (the message text is shortened by the test).

6.5 Inspect the Containers and the Network (Steps 18-25)

Concept: These steps check the deployment configuration: non-root users, restart policy, health status, the private network, the environment variable that connects the UI to the API, the separation of components and the resource use.

Step 18: User inside both containers (should not be root) (exit code 0)

```
$ docker exec dl-api id && docker exec dl-ui id
```

```
uid=10001(appuser) gid=10001(appuser) groups=10001(appuser)
uid=10001(appuser) gid=10001(appuser) groups=10001(appuser)
```

Observation: The processes of both containers run as uid 10001 (appuser), not as root.

Step 19: Container configuration and health (exit code 0)

```
$ docker inspect dl-api dl-ui --format '{{.Name}}: User={{.Config.User}} | Health={{.State.Health.Status}} |
  Image={{.Config.Image}} | Restart={{.HostConfig.RestartPolicy.Name}}'
```

```
/dl-api: User=appuser | Health=healthy | Image=dl-api:1.0 | Restart=unless-stopped
/dl-ui: User=appuser | Health=healthy | Image=dl-ui:1.0 | Restart=unless-stopped
```

Observation: docker inspect confirms for both containers User=appuser, Health=healthy and the restart policy unless-stopped.

Step 20: Containers attached to the private network (exit code 0)

```
$ docker network inspect dl-app-net --format '{{range .Containers}}{{.Name}} ->
  {{.IPv4Address}}{{println}}{{end}}'
```

```
dl-api -> 172.19.0.2/16
dl-ui -> 172.19.0.3/16
```

Observation: The private network dl-app-net contains dl-api (172.19.0.2/16) and dl-ui (172.19.0.3/16).

Step 21: API address configured in the UI container (exit code 0)

```
$ docker exec dl-ui printenv API_URL
```

```
http://api:5000
```

Observation: The UI container is configured with API_URL=http://api:5000, i.e. the service name of the API container.

Step 22: Installed components: api container (exit code 0)

```
$ docker exec dl-api python -c "import importlib.util as u; print('flask installed:', u.find_spec('flask') is
  not None); print('streamlit installed:', u.find_spec('streamlit') is not None); print('numpy installed:',
  u.find_spec('numpy') is not None)"
```

```
flask installed: True
streamlit installed: False
numpy installed: True
```

Observation: The api container contains Flask and NumPy but not Streamlit.

Step 23: Installed components: ui container (exit code 0)

```
$ docker exec dl-ui python -c "import importlib.util as u; print('flask installed:', u.find_spec('flask') is not
   None); print('streamlit installed:', u.find_spec('streamlit') is not None); print('numpy installed:',
  u.find_spec('numpy') is not None)"
```

```
flask installed: False
streamlit installed: True
numpy installed: True
```

Observation: The ui container contains Streamlit (and NumPy as a dependency of Streamlit) but not Flask. Each container holds only what it needs.

Step 24: Container logs (last lines of both services) (exit code 0)

```
$ docker compose logs --tail 6
```

```
dl-api  | 172.19.0.3 - - [10/Oct/2026:09:32:16 +0000] "GET /health HTTP/1.1" 200 38 "-" "python-requests/2.34.2"
dl-api  | 172.19.0.3 - - [10/Oct/2026:09:32:16 +0000] "GET /model-info HTTP/1.1" 200 433 "-" "python-
  requests/2.34.2"
dl-api  | 172.19.0.3 - - [10/Oct/2026:09:32:16 +0000] "GET /health HTTP/1.1" 200 38 "-" "python-requests/2.34.2"
dl-api  | 172.19.0.3 - - [10/Oct/2026:09:32:16 +0000] "GET /model-info HTTP/1.1" 200 433 "-" "python-
  requests/2.34.2"
dl-api  | 172.19.0.3 - - [10/Oct/2026:09:32:16 +0000] "POST /predict HTTP/1.1" 400 70 "-" "python-
  requests/2.34.2"
dl-api  | 127.0.0.1 - - [10/Oct/2026:09:32:16 +0000] "GET /health HTTP/1.1" 200 38 "-" "Python-urllib/3.12"
dl-ui   |   You can now view your Streamlit app in your browser.
dl-ui   |
dl-ui   |   Local URL: http://localhost:8501
dl-ui   |   Network URL: http://172.19.0.3:8501
dl-ui   |   External URL: http://<public-IP>:8501
dl-ui   |
```

Observation: The gunicorn access log of dl-api shows requests from 172.19.0.3, the UI container (user agent python-requests): GET /health, GET /model-info and a POST /predict answered with 400 from the invalid-input test; the line from 127.0.0.1 with Python-urllib is the HEALTHCHECK. The Streamlit log shows the Local, Network and External URL (the public address is masked in this report).

Step 25: Resource usage of both containers (exit code 0)

```
$ docker stats --no-stream dl-api dl-ui
```

```
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT     MEM %     NET I/O         BLOCK I/O     PIDS
a8ed3d01a5bd   dl-api    0.02%     122MiB / 7.748GiB     1.54%     109kB / 112kB   62.4MB / 0B   22
bd8dc373b07b   dl-ui     0.59%     53.38MiB / 7.748GiB   0.67%     56.3kB / 49kB   2.79MB / 0B   6
```

Observation: dl-api used 0.02 % CPU and 122 MiB of memory with 22 processes (gunicorn); dl-ui used 0.59 % CPU and 53.38 MiB with 6 processes. The API reading was taken shortly after the test runs.

6.6 Resilience and Dependency Checks (Steps 26-28)

Concept: The API container is restarted, stopped and started again to see whether the UI survives and whether the application recovers without manual intervention.

Step 26: Restart the API container; the UI can reach it again (exit code 0)

```
$ docker restart dl-api && for i in $(seq 1 60); do s=$(docker inspect --format '{{.State.Health.Status}}' dl-
  api); echo "check $i: $s"; [ "$s" = healthy ] && break; sleep 2; done; [ "$s" = healthy ] && docker compose
  exec -T ui python -c "import urllib.request;
  print(urllib.request.urlopen('http://api:5000/health').read().decode())"
```

```
dl-api
check 1: starting
check 2: starting
check 3: starting
check 4: healthy
{"status":"ok","uptime_seconds":6.0}
```

Observation: After docker restart the API passed through starting again and became healthy at check 4. From the UI container /health answered ok with an uptime of 6.0 s, so the UI reaches the restarted API without any change.

Step 27: Stop the API; the UI container keeps running (exit code 0)

```
$ docker compose stop api && sleep 2 && curl -s http://localhost:8501/_stcore/health && echo && docker ps -a
  --filter name=dl- --format 'table {{.Names}}\t{{.Status}}'
```

```
ok
NAMES     STATUS
dl-ui     Up 22 seconds (healthy)
dl-api    Exited (0) 2 seconds ago

[stderr]
 Container dl-api Stopping
 Container dl-api Stopped
```

Observation: After docker compose stop api the container dl-api is Exited (0) while dl-ui stays Up (healthy) and its health endpoint still answers ok: the UI does not crash when the API is down (see also Figure 14).

Step 28: Start the API again and repeat the UI integration test (exit code 0)

```
$ docker compose start api && for i in $(seq 1 60); do s=$(docker inspect --format '{{.State.Health.Status}}'
  dl-api); echo "check $i: $s"; [ "$s" = healthy ] && break; sleep 2; done; [ "$s" = healthy ] && docker compose
   exec -T ui python - < tests/ui_apptest.py 2>&1 | grep -v -i 'warning\|ScriptRunContext' | tail -4
```

```
check 1: starting
check 2: starting
check 3: starting
check 4: healthy
[PASS] Invalid input shows the API error message in the UI: ['Prediction failed: Each image needs exactly 64
  pixel values (flattened 8x8).']
[PASS] UI shows an error (no crash) when the API is unreachable: API not reachable:
  HTTPConnectionPool(host='127.0.0.1', port=9): Max r

8/8 UI integration checks passed

[stderr]
 Container dl-api Starting
 Container dl-api Started
```

Observation: After docker compose start api the API became healthy again (check 4) and the UI integration test passed once more (8/8; the last four lines are shown), so the whole application recovers after an API outage.

6.7 Cleanup (Step 29)

Step 29: Stop and remove the containers and the network (images are kept) (exit code 0)

```
$ docker compose down && docker ps -a --format 'table {{.Names}}\t{{.Status}}'
```

```
NAMES     STATUS

[stderr]
 Container dl-ui Stopping
 Container dl-ui Stopped
 Container dl-ui Removing
 Container dl-ui Removed
 Container dl-api Stopping
 Container dl-api Stopped
 Container dl-api Removing
 Container dl-api Removed
 Network dl-app-net Removing
 Network dl-app-net Removed
```

Observation: docker compose down stopped and removed both containers and the network dl-app-net; docker ps -a lists only the header row. The images are kept.

7. Screenshots and Observations

All screenshots were captured on the author's Mac (Docker Desktop v4.94.0) while the application was running: terminal output, the Streamlit UI and the API in the browser, and Docker Desktop. Terminal screenshots are shown as taken; the two Docker Desktop screenshots are followed by an enlarged detail.

Notes for transparency.

- The runner script removes its containers in its last step (docker compose down). For the screenshots the application was therefore started again by hand with `docker compose up -d --build`. The container IDs in the screenshots (for example 6e80ef442d76 and b6be105998f1) differ from the scripted run in the log (a8ed3d01a5bd and bd8dc373b07b).
- The second build (Figure 1) reused every layer (CACHED) but produced new image IDs (dl-api c05c9ee12d06, dl-ui c0bb52c13b5f) instead of d081e9baa059 and a213b7a30774 of the scripted build. The Dockerfiles are the same and the sizes are identical (303 MB and 779 MB).
- Figure 8 and `api_test_results.json` come from the manual run (server uptime 574.2 s); step 13 of the log is the scripted run. Both runs passed 13 of 13 checks.

7.1 Building and Starting the Application

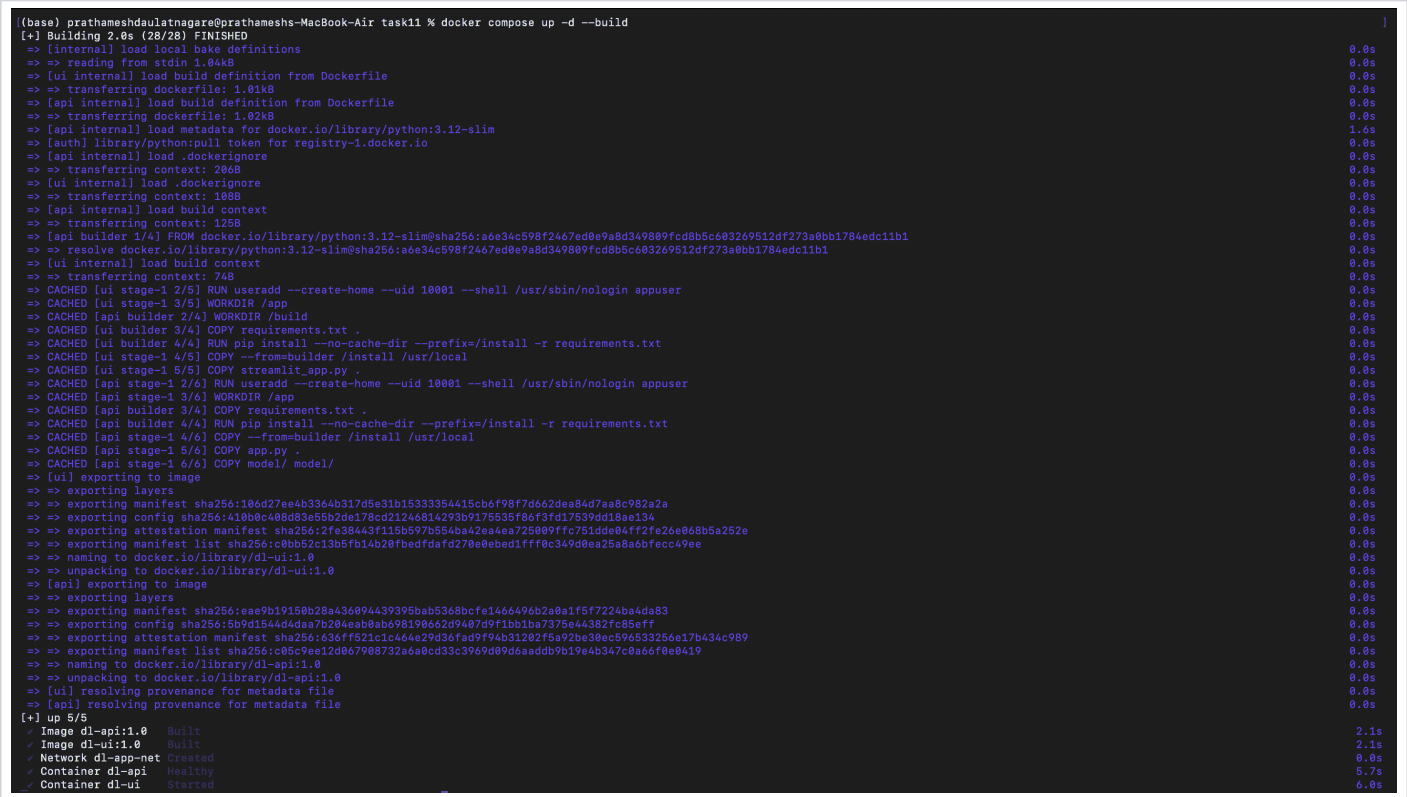


Figure 1: docker compose up -d --build: build of both images, creation of the network and start of the containers.

What the screenshot shows

- The build finished (28/28 steps) in 2.0 s because every layer was CACHED from the scripted run.
- Result lines: Image dl-api:1.0 Built, Image dl-ui:1.0 Built, Network dl-app-net Created.
- Container dl-api is reported Healthy (5.7 s) before Container dl-ui is Started (6.0 s): the UI waits for the API (depends_on, service_healthy).

7.2 Container Status

```
(base) prathameshdaulatnagare@prathameshs-MacBook-Air task11 % docker compose ps
NAME      IMAGE         COMMAND                  SERVICE   CREATED         STATUS                   PORTS
dl-api    dl-api:1.0    "gunicorn --bind 0.0…"   api       4 minutes ago   Up 4 minutes (healthy)   0.0.0.0:8000->5000/tcp, [::]:8000->5000/tcp
dl-ui     dl-ui:1.0     "streamlit run strea…"   ui        4 minutes ago   Up 4 minutes (healthy)   0.0.0.0:8501->8501/tcp, [::]:8501->8501/tcp
```

Figure 2: docker compose ps about four minutes after the start.

What the screenshot shows

- dl-api (service api, image dl-api:1.0, command gunicorn) and dl-ui (service ui, image dl-ui:1.0, command streamlit run): both 'Up 4 minutes (healthy)'.
- Ports: 0.0.0.0:8000->5000/tcp for the API and 0.0.0.0:8501->8501/tcp for the UI (each also for IPv6).

7.3 Image Sizes

```
(base) prathameshdaulatnagare@prathameshs-MacBook-Air task11 % docker image ls --filter 'reference=dl-*'

IMAGE                          ID             DISK USAGE   CONTENT SIZE   EXTRA
dl-api:1.0                     c05c9ee12d06       303MB         65.7MB   U
dl-app:1.0                     34d8650d4c16       324MB         70.7MB
dl-deploy-api:latest           1d9646e0048f      2.28GB          421MB
dl-deploy-frontend:latest      6371e70a23bc       792MB          175MB
dl-digits:1.0                  3c11f996f9d5       603MB          130MB
dl-ui:1.0                      c0bb52c13b5f       779MB          172MB   U
```

Figure 3: docker image ls --filter 'reference=dl-'.*

What the screenshot shows

- dl-api:1.0: ID c05c9ee12d06, disk usage 303 MB, content size 65.7 MB, marked U (in use).
- dl-ui:1.0: ID c0bb52c13b5f, disk usage 779 MB, content size 172 MB, marked U (in use).
- The other dl-* images in the list (dl-app, dl-deploy-api, dl-deploy-frontend, dl-digits) belong to earlier exercises.

7.4 The Streamlit UI: Start Page

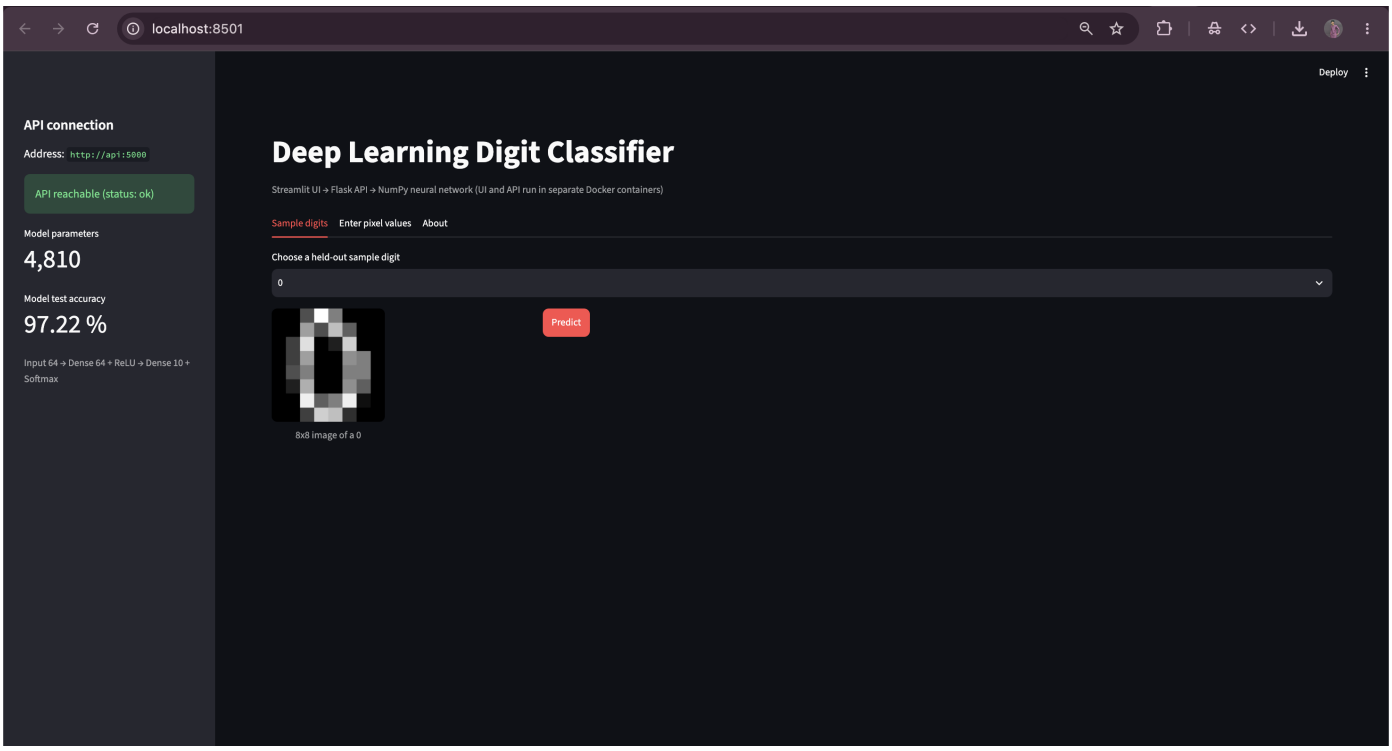


Figure 4: the Streamlit UI at http://localhost:8501 with the sample digit 0 selected, before pressing Predict.

What the screenshot shows

- Sidebar: API address http://api:5000, the green message 'API reachable (status: ok)', 4,810 model parameters, model test accuracy 97.22 % and the architecture line, all read from the API container.
- Tabs: Sample digits, Enter pixel values and About. The 8x8 image of a 0 and the Predict button are shown.

7.5 Prediction in the UI

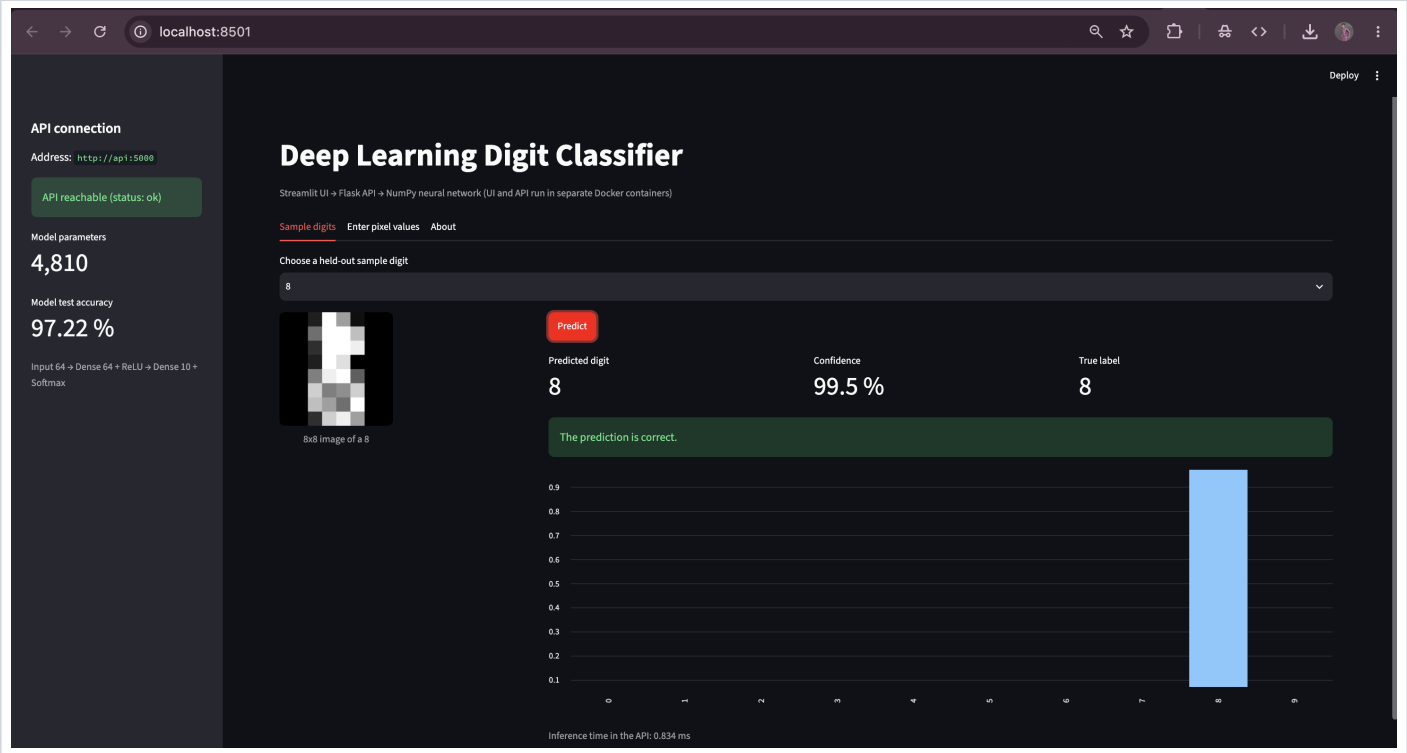


Figure 5: sample digit 8 after pressing Predict.

What the screenshot shows

- Predicted digit 8, confidence 99.5 %, true label 8, and the green message 'The prediction is correct.'
- The bar chart shows the probability for each digit with the peak at 8; the inference time reported by the API is 0.834 ms.

7.6 Prediction from Pixel Values

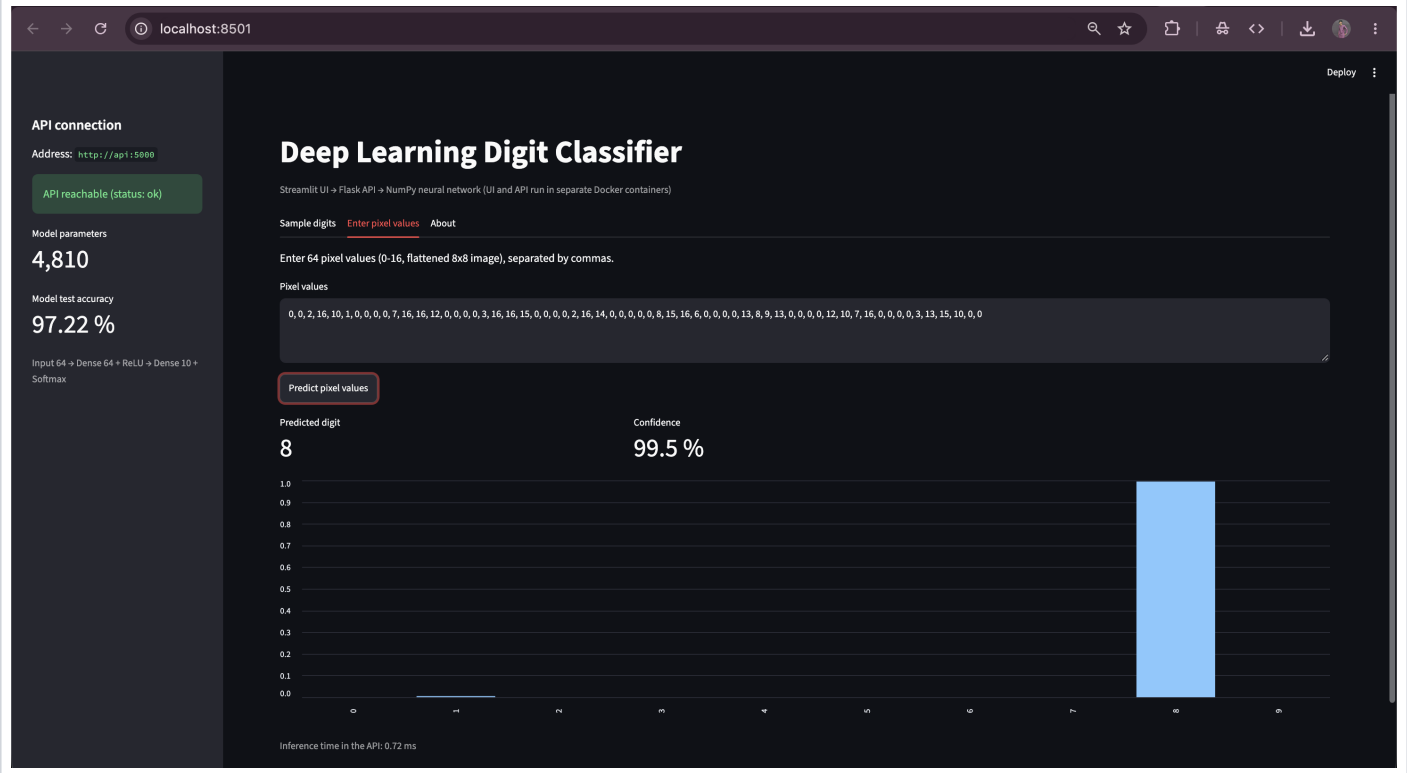


Figure 6: tab 'Enter pixel values' with 64 pre-filled pixel values and the result of 'Predict pixel values'.

What the screenshot shows

- The 64 comma-separated pixel values (flattened 8x8 image) are sent to POST /predict of the API.
- Result: predicted digit 8, confidence 99.5 %, inference time 0.72 ms.

7.7 The API Directly

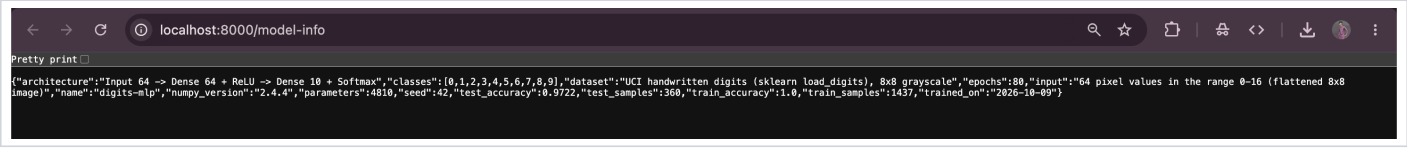


Figure 7: http://localhost:8000/model-info in the browser.

What the screenshot shows

- The API container answers directly on host port 8000 with the model metadata: 4,810 parameters, 1,437 training and 360 test samples, test accuracy 0.9722.

7.8 API Test Suite

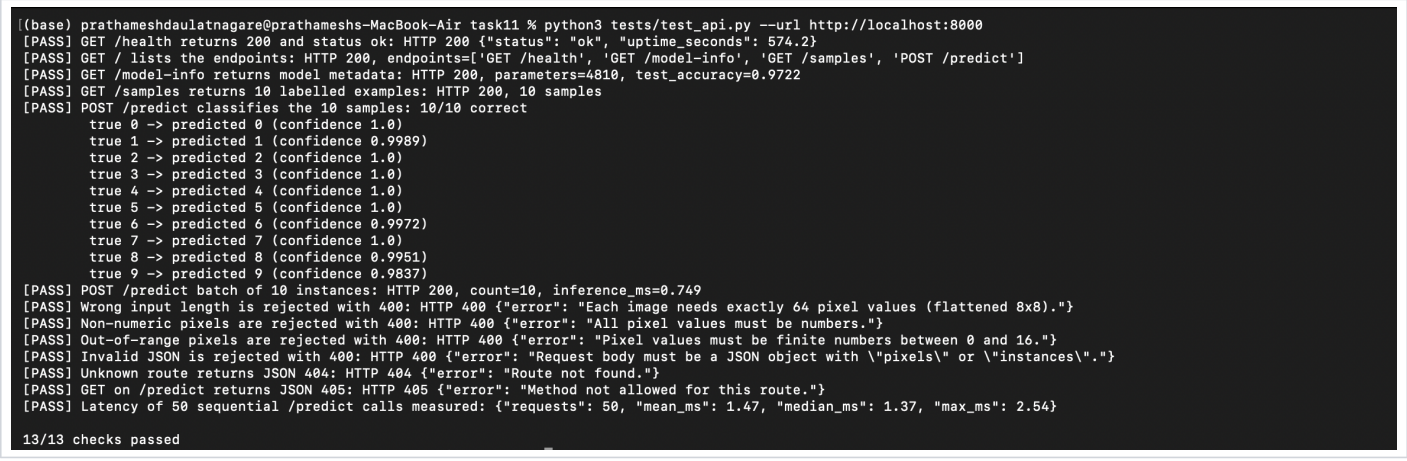


Figure 8: python3 tests/test_api.py --url http://localhost:8000 against the API container.

What the screenshot shows

- All 13 checks pass ('13/13 checks passed'); the server had been running for 574.2 s.
- 10 of 10 sample digits are classified correctly (lowest confidence 0.9837 for the digit 9); the batch of 10 took 0.749 ms.
- Invalid inputs return HTTP 400, an unknown route returns JSON 404, GET on /predict returns JSON 405; 50 sequential calls: mean 1.47 ms, median 1.37 ms, maximum 2.54 ms.

7.9 UI Integration Test inside the ui Container

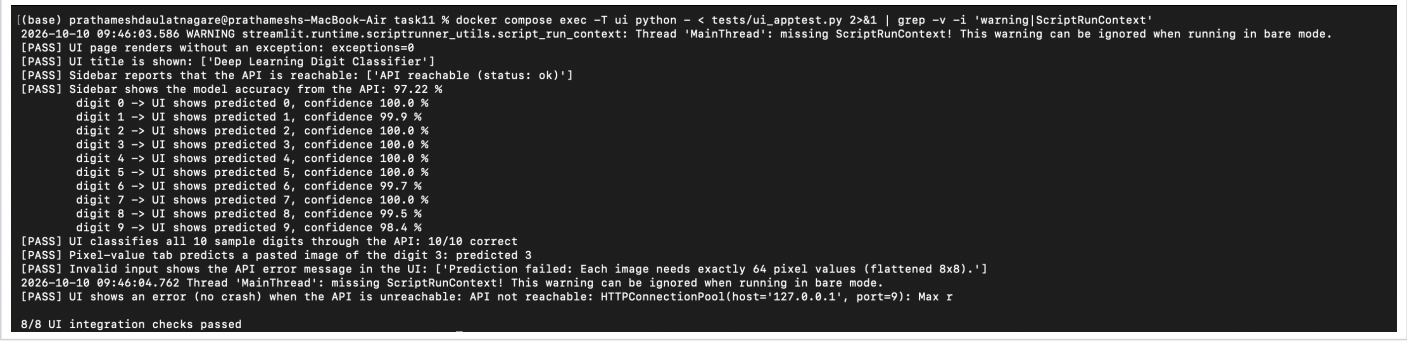


Figure 9: docker compose exec -T ui python - < tests/ui_apptest.py.

What the screenshot shows

- 8 of 8 UI integration checks pass, with the UI (running in the ui container) talking to the real API container.
- For each digit 0 to 9 the UI shows the correct predicted digit with its confidence (10/10).
- The two Streamlit warnings 'missing ScriptRunContext ... can be ignored when running in bare mode' appear because the grep pattern typed here did not filter them; they are harmless (the scripted run in step 17 filtered them).

7.10 Non-root Users and the Private Network

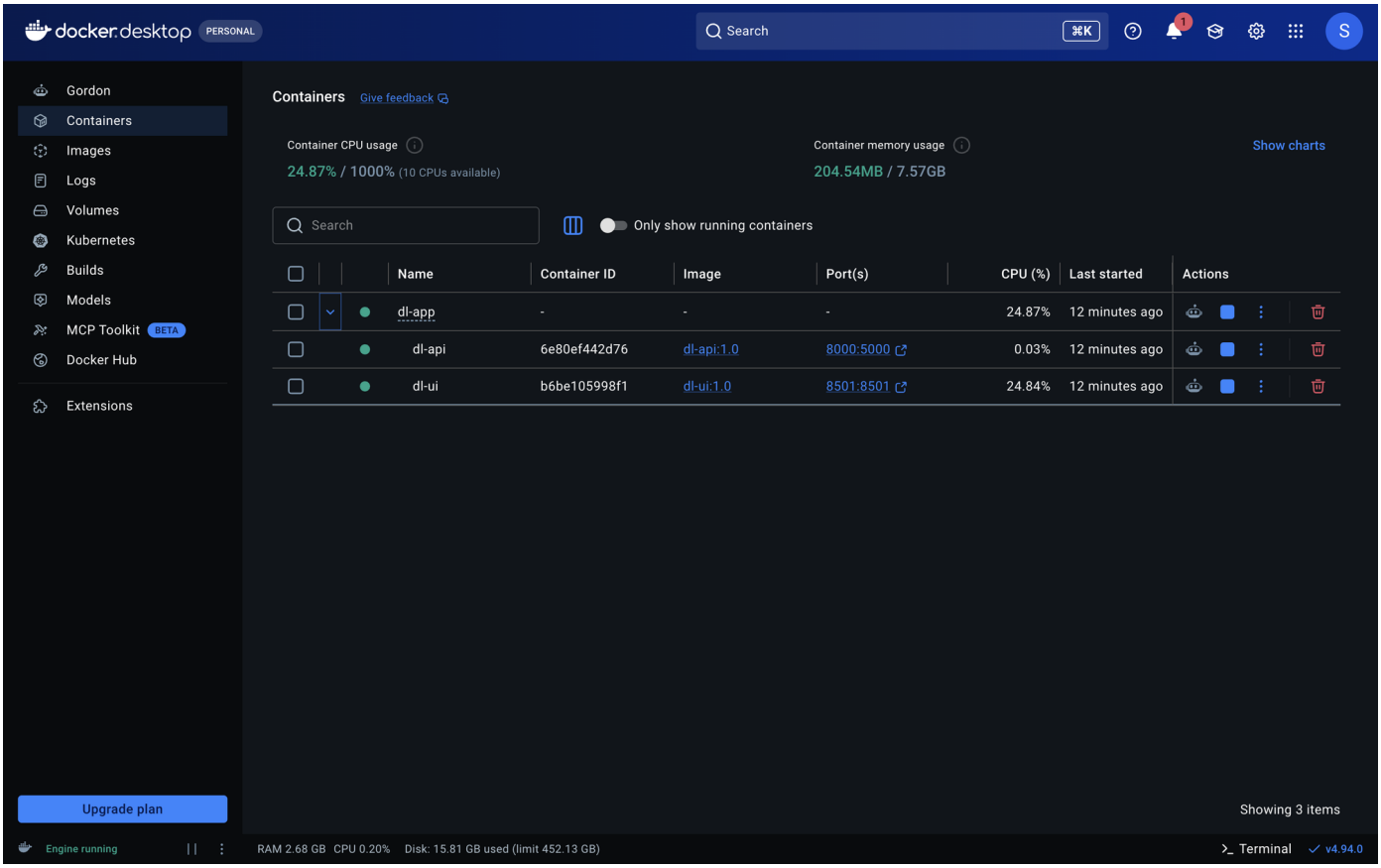
```
(base) prathameshdaulatnagare@prathameshs-MacBook-Air task11 % docker exec dl-ui id && docker exec dl-api id
uid=10001(appuser) gid=10001(appuser) groups=10001(appuser)
uid=10001(appuser) gid=10001(appuser) groups=10001(appuser)
(base) prathameshdaulatnagare@prathameshs-MacBook-Air task11 % docker network inspect dl-app-net --format '{{range .Containers}}{{.Name}} -> {{.IPv4Address}}{{println}}{{end}}'
dl-api -> 172.19.0.2/16
dl-ui -> 172.19.0.3/16
```

Figure 10: docker exec id for both containers, and docker network inspect dl-app-net.

What the screenshot shows

- Both containers run as uid=10001(appuser).
- The network dl-app-net contains dl-api (172.19.0.2/16) and dl-ui (172.19.0.3/16); the second line is cut off at the bottom of the screenshot and appears completely in step 20 of the log.

7.11 Docker Desktop - Containers



Enlarged detail of the same screen

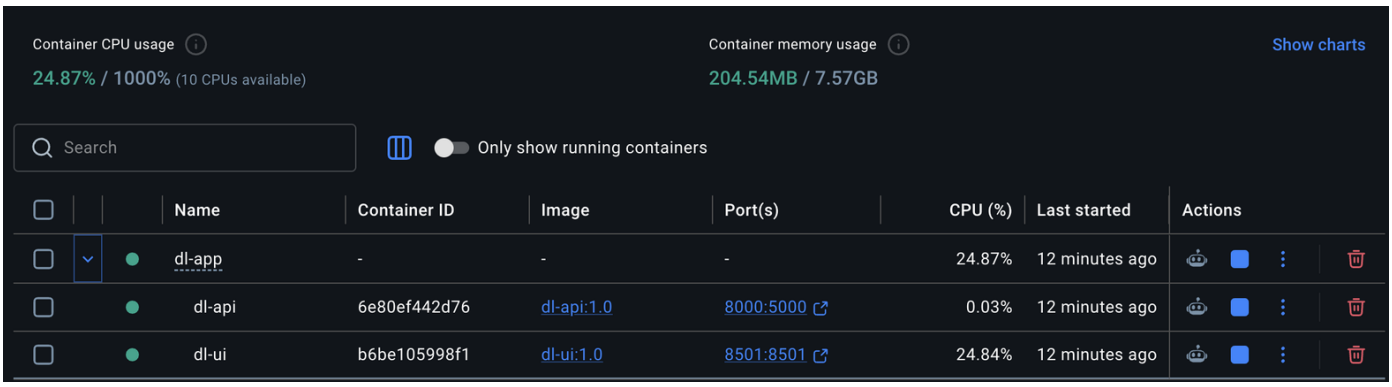
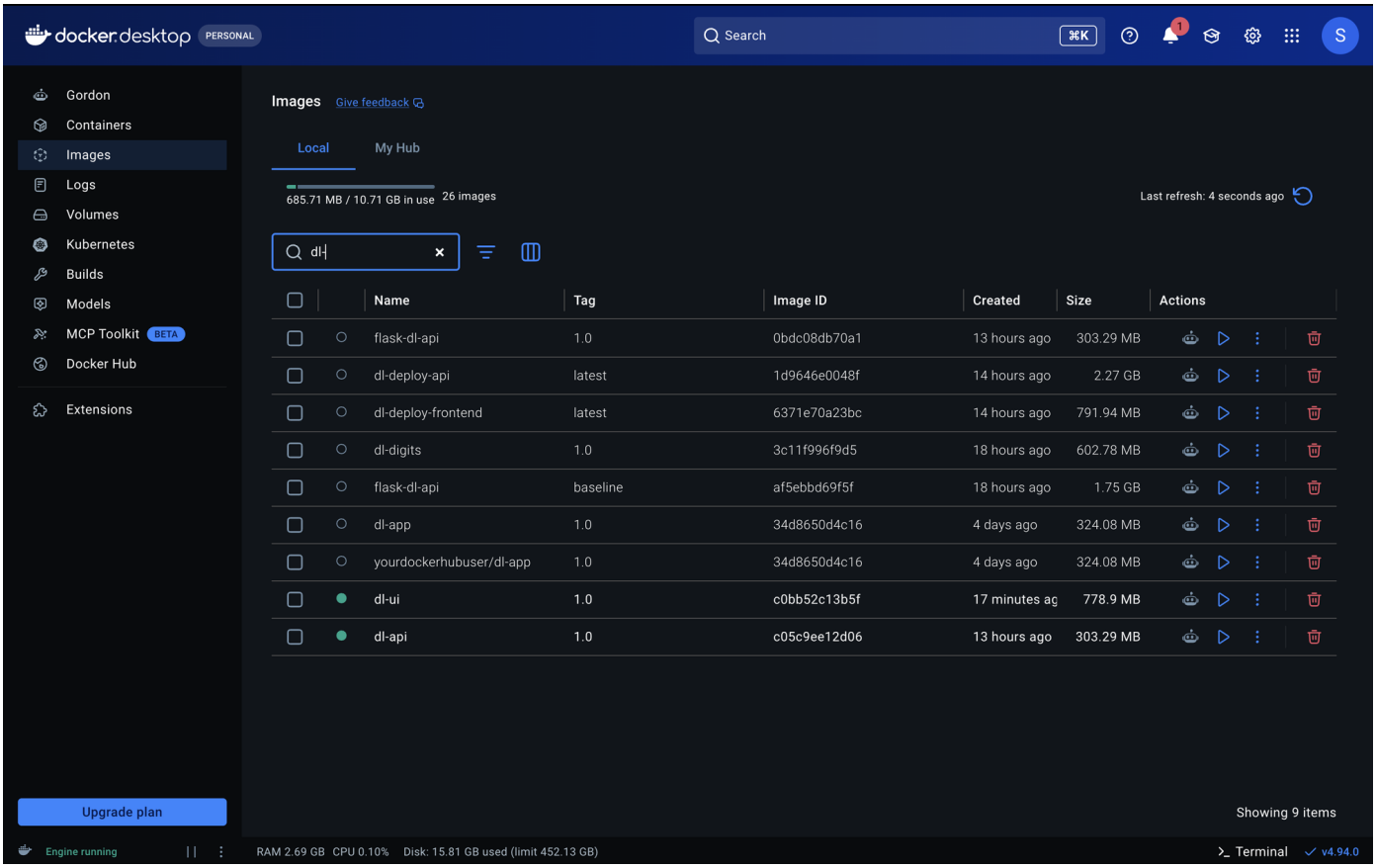


Figure 11: Docker Desktop, Containers tab, with the Compose group dl-app. The lower image enlarges the table.

What the screenshot shows

- The group dl-app contains dl-api (ID 6e80ef442d76, image dl-api:1.0, port 8000:5000, CPU 0.03 %) and dl-ui (ID b6be105998f1, image dl-ui:1.0, port 8501:8501, CPU 24.84 %); both are running, started 12 minutes ago.
- Summary at the top: container CPU 24.87 % of 1000 % (10 CPUs available), memory 204.54 MB of 7.57 GB; footer 'Engine running', Docker Desktop v4.94.0.

7.12 Docker Desktop - Images



Enlarged detail of the same screen

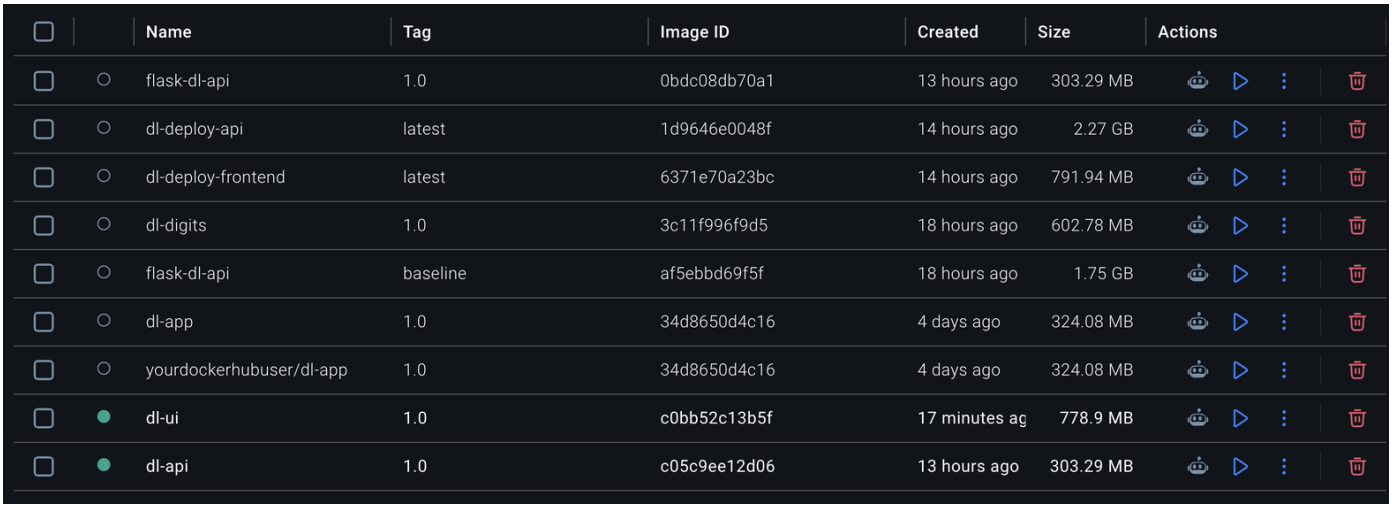


Figure 12: Docker Desktop, Images tab, filtered with 'dl-' (list refreshed 4 seconds earlier). The lower image enlarges the table.

What the screenshot shows

- dl-ui:1.0 (ID c0bb52c13b5f, 778.9 MB) and dl-api:1.0 (ID c05c9ee12d06, 303.29 MB) have a green dot: they are used by the running containers.
- The other entries are images from earlier exercises (for example flask-dl-api and dl-digits). dl-api is shown as created 13 hours ago because its layers were reused from earlier builds of the same Dockerfile.

7.13 Stopping the API Container

```
(base) prathameshdaulatnagare@prathameshs-MacBook-Air task11 % docker compose stop api
[+] stop 1/1
 ✔ Container dl-api Stopped
```

Figure 13: docker compose stop api.

What the screenshot shows

- Container dl-api is Stopped while the ui container keeps running (resilience test).

7.14 UI Behaviour while the API Is Down



Figure 14: the UI at http://localhost:8501 after the API container was stopped.

What the screenshot shows

- The UI does not crash: the sidebar shows the red message 'API not reachable' with the reason (the name 'api' cannot be resolved because the API container is stopped), and the page title is still displayed.
- After docker compose start api the application works again (step 28 of the log).

8. Testing Report

The application was tested on several levels: build, startup, API, UI, container integration, deployment configuration and resilience. All tests ran against the Docker containers (not against a local development server); the commands and outputs are in Section 6 and the screenshots in Section 7.

8.1 Test summary

Test level	What was tested	Where	Result
Build	both images build from their Dockerfiles with docker compose	steps 3-8, Fig. 1, 3	PASS
Startup and health	both containers become (healthy); the UI starts after the API	steps 9-12, Fig. 1, 2	PASS
API (black box)	13 checks: routes, 10 sample digits, batch, 4 invalid inputs, JSON 404/405, latency	step 13, Fig. 8	13/13 PASS
UI availability	health endpoint answers ok, main page returns HTTP 200	steps 14-15	PASS
Container integration	the ui container reaches the API by service name	step 16	PASS
UI integration	8 checks with the UI in the ui container calling the real API (AppTest)	step 17, Fig. 9	8/8 PASS
Manual UI test	sample digit and pixel-value prediction in the browser	Fig. 4-6	PASS
Deployment configuration	non-root users, restart policy, environment variable, network, separated components	steps 18-23, Fig. 10	PASS
Resilience	API restart, API outage and recovery; UI integration test repeated	steps 26-28, Fig. 13-14	PASS

8.2 API test results (api_test_results.json)

Test	Result
GET /health returns 200 and status ok	PASS
GET / lists the endpoints	PASS
GET /model-info returns model metadata	PASS
GET /samples returns 10 labelled examples	PASS
POST /predict classifies the 10 samples	PASS
POST /predict batch of 10 instances	PASS
Wrong input length is rejected with 400	PASS
Non-numeric pixels are rejected with 400	PASS
Out-of-range pixels are rejected with 400	PASS
Invalid JSON is rejected with 400	PASS
Unknown route returns JSON 404	PASS
GET on /predict returns JSON 405	PASS
Latency of 50 sequential /predict calls measured	PASS

8.3 UI integration test results (ui_integration_test.txt)

Test	Result	Detail
UI page renders without an exception	PASS	exceptions=0
UI title is shown	PASS	['Deep Learning Digit Classifier']
Sidebar reports that the API is reachable	PASS	['API reachable (status: ok)']
Sidebar shows the model accuracy from the API	PASS	97.22 %
UI classifies all 10 sample digits through the API	PASS	10/10 correct
Pixel-value tab predicts a pasted image of the digit 3	PASS	predicted 3
Invalid input shows the API error message in the UI	PASS	['Prediction failed: Each image needs exactly 64 pixel values (flattened 8x8).']
UI shows an error (no crash) when the API is unreachable	PASS	API not reachable: HTTPConnectionPool(host='127.0.0.1', port=9): Max r

8.4 Sample predictions through the API and through the UI

True digit	API predicted	API confidence	UI predicted	UI confidence
0	0	1.0000	0	100.0 %
1	1	0.9989	1	99.9 %
2	2	1.0000	2	100.0 %
3	3	1.0000	3	100.0 %
4	4	1.0000	4	100.0 %
5	5	1.0000	5	100.0 %
6	6	0.9972	6	99.7 %
7	7	1.0000	7	100.0 %
8	8	0.9951	8	99.5 %
9	9	0.9837	9	98.4 %

8.5 Resilience tests (steps 26-28)

Action	Observed	Result
docker restart dl-api	healthy again at check 4; the UI container reaches /health (uptime 6.0 s)	PASS
docker compose stop api	dl-api Exited (0); dl-ui stays Up (healthy) and /_stcore/health answers ok; the UI shows 'API not reachable' (Fig. 14)	PASS
docker compose start api	healthy again at check 4; UI integration test passes 8/8 again	PASS

8.6 Images, resources and results by approach area

Item	Value
Commands executed / succeeded	29 / 29
Image dl-api:1.0	303,293,481 bytes (303.3 MB); content size 65.7 MB
Image dl-ui:1.0	778,900,479 bytes (778.9 MB); content size 172 MB
Time until healthy	API: first check (Compose waited for it); UI: about 6 seconds (fourth check)
API latency (50 calls)	scripted run: mean 1.56 ms, median 1.51 ms, max 2.51 ms; manual run: mean 1.47 ms, median 1.37 ms, max 2.54 ms
Resources (step 25)	dl-api: 0.02 % CPU, 122 MiB, 22 processes; dl-ui: 0.59 % CPU, 53.38 MiB, 6 processes

Approach area	Steps	Succeeded
Verify Docker and Compose	1-2	2/2
Validate the Configuration and Build the Images	3-8	6/6
Start the Full Application with Docker Compose	9-12	4/4
Test the Application	13-17	5/5
Inspect the Containers and the Network	18-25	8/8
Resilience and Dependency Checks	26-28	3/3
Cleanup	29	1/1

9. Deployment Readiness

The table reviews whether the containerized application is ready to be deployed, with the evidence from the recorded run.

Criterion	How it is met	Evidence
One-command deployment	docker compose up -d --build builds and starts both containers and the network	Fig. 1, step 9
Reproducible builds	all dependencies pinned (9 and 36 packages), fixed base image python:3.12-slim	Sections 5.9-5.10
Correct start order	UI starts only after the API is (healthy)	step 9, Fig. 1
Health monitoring	HEALTHCHECK in both images; Docker reports (healthy)	steps 10-12, Fig. 2
Self-healing	restart policy unless-stopped; the application recovers after an API restart or outage	steps 19, 26-28
Configuration without code changes	the API address is set by the environment variable API_URL	steps 21, 16
Least privilege	both containers run as uid 10001 (appuser)	step 18, Fig. 10
Separation of concerns	the UI image has no Flask or model, the API image has no Streamlit	steps 22-23
Small images	slim base, multi-stage build, no pip cache; API 303.3 MB, UI 778.9 MB	steps 6-8, Fig. 3
Network isolation	containers talk over the private network dl-app-net; only ports 8000 and 8501 are published	steps 20, 12

Criterion	How it is met	Evidence
Observability	access log of the API and the Streamlit log are available with docker compose logs	step 24
Verified functionality	13/13 API checks, 8/8 UI integration checks, manual UI tests	Section 8

10. Observations

- **The full solution runs as two cooperating containers.** Compose created the network, started the API first and the UI after the API was healthy; the whole application was usable about 6 seconds after the UI container started.
- **The integration works through service names.** The UI container resolves the name api and its requests appear in the API's access log with the UI container's address (172.19.0.3), so no host port is needed for the UI-to-API traffic.
- **The tests pass at every level.** 13 of 13 API checks, 8 of 8 UI integration checks (also after the API was restarted), 10 of 10 sample digits correct through both the API and the UI.
- **The application is resilient.** With the API stopped the UI container stays up (healthy) and shows 'API not reachable' instead of crashing; after the API is started again everything works without touching the UI.
- **The images are separated and unprivileged.** Flask is only in the API image, Streamlit only in the UI image, and both run as appuser (uid 10001).
- **The UI image is much larger than the API image.** 778.9 MB against 303.3 MB: the packages installed for Streamlit (pandas, pyarrow, NumPy and others) alone form a 448 MB layer, while the API needs only Flask, gunicorn and NumPy.
- **Resource use is low for the API and moderate for the UI.** In step 25 dl-api used 122 MiB and dl-ui 53.38 MiB at almost no CPU load; in the Docker Desktop screenshot (Figure 11), taken with a browser session open, dl-ui showed 24.84 % CPU (of 1000 % available), which was not investigated further.
- **Both images could be built and run with one command.** docker compose up -d --build also reused the cached layers, so the second build took only 2.0 s.

11. Known Limitations

- **Small demonstration model.** The network is a small MLP on 8x8 digit images; it is not a production model and runs on the CPU only. The API accepts only flattened 64-value pixel vectors, not image files.
- **Large UI image.** The UI image is 778.9 MB because of Streamlit's dependencies; a lighter UI framework or a different packaging would reduce it.
- **Limited load testing.** Latency was measured with 50 sequential requests from the same machine; concurrent users and long-running behaviour were not tested.
- **Basic deployment setup.** The setup is for a single host: no HTTPS, no authentication, no reverse proxy, no image scanning and no registry push; orchestration with Kubernetes or Swarm was not used.
- **Architecture-specific numbers.** The images are arm64 (Apple Silicon); sizes and IDs differ on other machines, and the base-image tag python:3.12-slim changes over time.
- **UI CPU load not investigated.** The UI container showed a high CPU reading in one Docker Desktop screenshot while a browser session was open; the cause was not analysed.

- **Screenshots from a second run.** Container and image IDs in the screenshots differ from the log (see Section 7); one screenshot (Figure 10) cuts off the last network line, which is complete in step 20 of the log.

12. Conclusion

The complete deep-learning solution was containerized: a NumPy model and a Flask API in the dl-api image, a Streamlit UI in the dl-ui image, and a Docker Compose file that builds, networks and starts both. The UI reaches the API by service name over the private network dl-app-net, starts only after the API is healthy, and handles an API outage without crashing. Both containers run as a non-root user and have health checks. The testing report shows 13 of 13 API checks, 8 of 8 UI integration checks, 10 of 10 sample digits classified correctly through the API and the UI, successful restart and outage recovery tests, and 29 of 29 successful commands in the recorded run. With the architecture diagram, the Dockerfiles, the command log, the screenshots and the deployment-readiness review, this report documents a deployable, tested containerization of the full application.